Comparative Analysis of Conventional and Deep Learning Approaches in Forest Fire Detection

Deep Learning Project Report

Submitted by:
Vyshnavi Kanukuntla [CH.SC.U4AIE23025]
Naishadha Badithala [CH.SC.U4AIE23036]
Kaushik Poshimreddy [CH.SC.U4AIE23042]
Amrita Vishwa Vidyapeetham

Supervisor:
Dr. K. Deepak

September 15, 2025

Contents

List of Figures

List of Tables

1. Introduction

One of the most important things in computer vision is image feature extraction. Raw images consist of millions of pixels that are usually of high redundancy and high correlation. Processing these pixel data directly is usually inefficient, computationally expensive, and less effective for high-level vision tasks. Feature extraction creates a solution for this problem by converting pixel-level information into a compact, meaningful, and discriminative representation of data that retains the essential information in the image while reducing dimensionality.
Features extracted may represent edges, corners, textures, shapes, or even semantic-level concepts. The quality of features directly affects the performance of computer vision tasks such as object recognition, face verification, medical image analysis, scene understanding, and image retrieval. With the passage of time, feature extraction has metamorphosed from handcrafted descriptors that were created by human intuition to today-adays deep learning and transformer-based methods, wherein features are learned automatically from a massive amount of data.

1.1 Significance of Feature Extraction in Computer Vision

The paramount importance of feature extraction stems from its potentiality to select the most distinguishing facets of an image. By finding a reduced representation of raw pixel data that is more meaningful, feature extraction enables generalization, computation efficiency, and robustness under various situations-scaling, rotation, illumination-which can affect the system performance. Hand-crafted descriptors like SIFT and HOG tremendously influenced early-age object recognition because of their paramount ability to resist transformations.
In real-life applications, the extracted features are the performers across tasks. For example, pedestrian detection systems for surveillance work rely on the Histogram of Oriented Gradients (HOG), whereas SLAM systems in robotics rely mostly on ORB features to have real-time performance. In modern deep learning applications, features learned by CNNs are used in image classification, object detection, and retrieval, where they are often more robust and scalable when compared to hand-crafted descriptors. Hence, it can be said that the computer vision pipelines have feature extraction as their backbone.

1.2 Taxonomy of Image Feature Extraction

Feature extraction techniques can be broadly categorized into:

1. By scope of representation:
   Local features: Focus on specific keypoints (SIFT, ORB, SuperPoint).
   Global features: Summarize the entire image (BoVW, VLAD, CNN pooled vectors).

2. By design principle:
   Hand-crafted: Manually engineered descriptors (HOG, SIFT, LBP).
   Learned: Features obtained through supervised or unsupervised deep learning (CNN, ViT, DELF).

3. By descriptor type:
   Float descriptors: Continuous feature vectors (SIFT, CNN features).
   Binary descriptors: Compact binary strings for fast matching (BRIEF, ORB, BRISK).

4. By density:
   Sparse (keypoint-based): Extract features only at distinctive points.
   Dense: Generate features for every pixel or patch (CNN feature maps, D2-Net).

1.3 Classical Hand-Crafted Features

Early-stage feature extraction research was concerned with finding the descriptors for the local pattern of images. One of the oldest and successful techniques was the Harris Corner Detector, which considered a corner to be the actual point of interest, mainly due to its stability under transformations. Later at one landmark moment, the SIFT algorithm extracts features that are invariant to scale/rotation/illumination change and thus gained prominence in recognition applications. SURF was introduced to speed up the whole process while attempting to keep its robustness intact using integral images.
Other frequently used highly attractive descriptors include HOG, for it encodes directions of edges in spatial regions and has been shown to be very effective for human detection, and LBP, for it captures texture patterns and has been under-the-radar-famously successful in face recognition. In addition, Gabor filters afforded multi-scale and multi-orientation analyses of textures and proved to be useful in biometrics.
Although hand-crafted features were interpretable and efficient, some limitations were present. Although they were based on human-designed rules, they were rather unadaptive to complex permutations of the data, and hence they lacked the semantics required for large-scale vision tasks.

1.4 Binary Descriptors and Efficiency-Oriented Methods

As applications soon started requiring an online setup, mainly in robotics and mobile vision systems, lightweight binary descriptors came into existence. The Binary Robust Independent Elementary Features (BRIEF) grew a compact representation by comparing pixel intensities

into binary string encodings. ORB (Oriented FAST and Rotated BRIEF) took this further by combining a corner detector that was fast with rotational invariance, making it one of the premiers in SLAM and augmented reality because of its speed and accuracy. From there, BRISK and FREAK further evolved this concept by attempting to enhance scale invariance and robustness but still keeping the descriptors compact.
These descriptors revolutionized applications where computational resources were limited. However, their compactness often came at the cost of reduced discriminative power compared to float-based descriptors like SIFT.

1.5 Global Image Representations

Local descriptors capture features around keypoints, whereas global descriptors eschew such localism and summarize an entire image instead. The first successful global approach was BoVW, which represents the image as a histogram of clustered local features. This was further extended by SPM, which retained partial spatial information by subdividing images into grids. Further developments include Vector of Locally Aggregated Descriptors (VLAD) and Fisher Vector (FV), which exploited encoding not only feature existence but also statistical information on feature distribution to gain representation-level power. On the global level, considerable image retrieval tasks were performed well by these representations. However, they failed in fine-grained matching due to their incompatibility with local discrepancies on a precise level.

1.6 Deep Learning-Based Features

The introduction of deep learning is considered a paradigm shift with respect to feature extraction. In 2012, the success of AlexNet demonstrated the ability of CNNs to learn features automatically from raw pixel data, provided there are sufficient labeled samples. Rather than a hand-crafted approach, CNNs learn representation hierarchically, starting from low-level edges in the early layers to complex semantic structures in the deeper ones.
In providing deep feature representation, architectures such as VGGNet and ResNet were considered a few "deep feature" extractors since the features they uncover are truly transferable, in the sense that they can be generalized across multiple tasks. CNN-based features soon became the default choice for classification, object detection, and retrieval, thereby replacing traditional descriptors in most fields.

Let's learn how CNN works with patch-based classification. When an image is fed into the system, a small contiguous patch is extracted from it; after it ispassed through the CNN, it is classified as either containing the object or not. The offset of the patch is recorded against the classification."
A deep feature's prowess comes from its capability to grasp information from data adaptively and to capture semantic information with some robustness to variation. For concrete examples of applications, deep features have been used in medical image-based disease classification,

object identification in self-driving cars, and large-scale retrieval in visual search systems.

1.7 Learned Local Features

To overcome the barrier posed by hand-made keypoint descriptors, it started the era for learning local features. One of the first frameworks to learn detection, orientation, and description simultaneously and in an end-to-end way was LIFT (Learned Invariant Feature Transform). SuperPoint proposed a self-supervised method to obtain reliable interest points and descriptors without the need for large-scale labeled data, thus opening avenues for real-time application utilization.
Later, dense local feature extraction using a deep network was proposed by D2-Net, whereas DELF introduced attention to emphasize the most discriminative image parts for retrieval. These methods showed that learned local features could perform better than traditional descriptors with robustness and scalability, especially in landmark recognition and large-scale retrieval problems.

1.8 Self-Supervised and Contrastive Learning Approaches

An excellent recent advancement has been self-supervised learning, which alleviates the need for large annotated datasets. Methods like SimCLR and MoCo contrast the representation of augmented views of the same image with other images, while BYOL does not require negative pairs to obtain similar results. More recently, DINO established that Vision Transformers (ViTs) trained with self-supervision can generate semantic features that correlate well with the objects in the images, even without any labels.
In fields where labeled data is scarce, like medical imagery or satellite images, such methods become super important as they allow feature extraction from massive unlabeled datasets, standing competitively with supervised ones.

1.9 Transformer-Based Feature Extraction

Being natural language processing paramour, ViT has recently been adjusted for vision tasks. The original idea behind the Vision Transformer was to treat images as sequences of patches and to apply self-attention to capture global dependencies. Going contrary to CNNs, whose fundamental assumed paradigm is local receptive fields, transformers go ahead and model long-range interactions with a natural intuition. The Swin Transformer improved this method by introducing a hierarchical architecture with shifted windows, thereby making it efficient for dense prediction tasks such as segmentation.
Transformer-based features that are most of the time generated using self-supervised approaches have set new image classification, retrieval, and dense matching baselines. Their ability to

understand both global and local contexts makes them very promising for the future of feature extraction.

1.10 Conclusion

Image feature extraction has seen an evolution over time. Earlier methods like SIFT, HOG, and LBP provided interpretable and efficient representations but lacked semantic richness. The introduction of binary descriptors enabled real-time performances, with the drawback of losing their discriminative power. Global descriptors such as BoVW and VLAD favor large-scale retrieval but suffer at fine-grained matching.

With the deep learning revolution, it got further towards learned features, with CNNs offering hierarchical, transferable representation that is dominating the present applications. In recent times, local feature learning, self-supervision, and transformer-based architecture paradigms have been sought to push those boundaries further to achieve robust and scalable feature extraction with the help of minimal manual design or extensive labeling process.

Looking ahead, the future of feature extraction lies in hybrid approaches that infuse the interpretability of classical methods with the flexibility of deep and transformer-based models to solve vision problems efficiently, robustly, and semantically across a wide range of applications.

2. Conventional Image Feature Extraction Methods

2.1 Histogram of Oriented Gradients (HOG)

2.1.1 Principle

The Histogram of Oriented Gradients (HOG) method is based on the idea that object appearance and shape can be effectively characterized by the distribution of edge directions (gradients). Instead of using raw pixel values, HOG captures how gradients are oriented across local regions of the image, making it robust to changes in illumination and small variations in pose.

2.1.2 Algorithm Workflow

1. Preprocessing: Normalize image intensity and resize.
2. Gradient Computation: Calculate G_x, G_y, magnitude M, and orientation θ.
3. Cell Histograms: Divide the image into small cells (e.g., 8×8 pixels) and compute gradient orientation histograms.
4. Block Normalization: Group neighboring cells into blocks (e.g., 2×2 cells) and normalize to reduce illumination effects.
5. Feature Vector Construction: Concatenate normalized histograms into a single feature descriptor.

2.1.3 Applications

- Human detection: Widely used in pedestrian detection systems.
- Object recognition: Cars, animals, and other objects in surveillance.
- Medical imaging: Detection of cell structures or anomalies.

2.2 Scale-Invariant Feature Transform (SIFT)

2.2.1 Principle

The Scale-Invariant Feature Transform (SIFT) extracts distinctive local keypoints from images that are invariant to scale, rotation, and partially invariant to illumination and affine transformations. Each keypoint is described by a high-dimensional vector, making it highly robust for matching across different views of the same scene or object.

2.2.2 Algorithm Workflow

1. Scale-Space Construction: Generate progressively blurred images using Gaussian filters.
2. Keypoint Detection: Detect extrema using Difference of Gaussians across scales.
3. Keypoint Localization: Refine detected points by removing low-contrast and edge responses.
4. Orientation Assignment: Assign a dominant orientation to each keypoint using local gradients.
5. Descriptor Generation: Create a 128-dimensional descriptor by building histograms of gradients around the keypoint.

2.2.3 Applications

- Image stitching: Matching overlapping features across multiple images.
- 3D reconstruction: Keypoint matching across different viewpoints.
- Object recognition and retrieval: Identifying objects in cluttered scenes.
- Robotics and SLAM: Landmark detection for navigation.

2.3 Gray-Level Co-Occurrence Matrix (GLCM)

2.3.1 Principle

Unlike HOG and SIFT, which are gradient or keypoint-based, the Gray-Level Co-Occurrence Matrix (GLCM) focuses on texture. It captures how often pairs of pixel intensities (gray levels) occur at a specified distance and orientation in an image. This statistical method provides second-order texture information and is especially useful in domains where surface patterns are important.

2.3.2 Algorithm Workflow

1. Quantization: Convert the image to a limited number of gray levels.
2. GLCM Construction: Compute co-occurrence frequencies at chosen distances (e.g., 1 pixel) and orientations (e.g., 0°, 45°, 90°, 135°).
3. Normalization: Normalize the matrix to probabilities.
4. Feature Extraction: Derive statistical measures (contrast, energy, homogeneity, entropy).
5. Feature Vector: Combine all derived measures into a descriptor for classification.

2.3.3 Applications

- Medical imaging: Tumor detection in MRI or CT scans.
- Remote sensing: Land cover classification using texture.
- Industrial inspection: Surface defect detection in manufacturing.
- Biometrics: Fingerprint and iris texture analysis.

2.4 Conclusion

Conventional feature extraction methods use different sets of image attributes. HOG takes the edge and gradient distributions into consideration and therefore is very useful in shape and contour detection. SIFT extracts scale- and rotation-invariant keypoints, thus becoming robust for matching under transformations. GLCM perceives texture statistics, which is, therefore, very suitable for medical imaging and pattern analysis.
Together, these techniques demonstrate how gradient-, keypoint-, and texture-based descriptors complement one another in various computer vision tasks. While modern deep learning approaches have taken center stage in nearly every vision pipeline today, the common and conventional methods still find usage in real-time systems, resource-constrained environments, or in applications requiring an interpretable framework for domain-specific analysis.

3. Experimentation

3.1 Dataset Selection

Initially, we searched for some appropriate set of data to work with. We finally found this Forest Fire dataset for Binary Classification, which was already split into two classes: fire and no fire. While our entire initial base was good enough, we felt that our model would be of little use when real-life forest fire detection requires it to do well under various conditions and scenarios. Going back to these issues, we extended and enriched the dataset by contributing our own fix thereto. The protocol involved manual collection of extra images from diverse online resources, including public datasets, open repositories, and other such web sources, to avoid any bias towards a specific image source or environment. The images we added consisted of the fire sort (active flames, smoke, wildfire outbreaks in forests) as well as nofire sorts (normal forest environments captured under different conditions). Hence, after merging the original dataset with ours, the entire set consisted of 6,248 files, giving us a sufficiently large image pool to train, validate, and adequately test models on.

The fire class was carefully created to equally represent the different kinds of wildfire happenings: raging flames, controlled burns, smoke plumes, etc., with differing intensities. These images constitute an international and environmental reception, for the model to be able to learn fire features that are truly robust, rather than over-fitting a very narrow dataset. To increase variability while keeping the core semantics intact, mild edits like random rotations or horizontal flips, plus a touch of brightness adjustment with some zoom variations, have been kept in place. These augmentations aid, to a certain extent, in imbibing very real-life variations of how fire could look on either a camera or a surveillance feed.

The nofire class, however, was intended to capture the natural diversity of forests without fire. We took images of forests across seasons, lighting conditions, and landscapes: lush greenery, dry vegetation, cloudy skies, and clear weather. The intent was to make a broad spectrum of non-fire situations so that the model would not associate usual forest textures with reflections of sunlight or haze as fire. As with the fire class, we performed light augmentations on the nofire images to prevent overtraining and allow better generalization.

Once the two classes were fixed, the data set was split into train-test-validation piles with 70-20-10 percentages. Such a treatment is common whereby we try to maximize the training data but keep some reference data to evaluate and tune hyperparameters in an unbiased manner. This

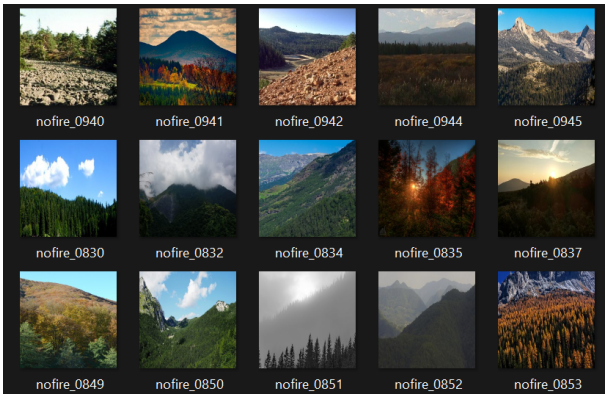


Figure 3.1: Sample: nofire

Figure 3.2: Sample: fire

way, the splitting of data was such that the model development methodology was determined and reproducible.

One of the far-reaching implications of the juxtaposition of the existing curated dataset with our manual additions and augmentation pipeline is that the dataset built is not only larger and more diversified but actually more paradigmatic of real-world conversion of wildfire detection challenges. Such an enriched dataset, on the other hand, truly highlights the varying forest conditions and fire appearances and thus becomes apt for training a deep learning model that has to perform the final tasks of high accuracy and robustness in actuality.

3.2 Traditional Feature Extraction

In order to evaluate the performance of traditional feature extraction methods on the forest fire dataset, four widely recognized techniques were chosen: Histogram of Oriented Gradients (HOG), Scale-Invariant Feature Transform (SIFT), Gray-Level Co-occurrence Matrix (GLCM), and Oriented FAST and Rotated BRIEF (ORB). Each of these methods was selected based on its ability to capture different aspects of image characteristics such as texture, edge orientation, shape, and local descriptors. Since the dataset contains complex scenes with fire and non-fire

images under varying illumination and background conditions, it is crucial to employ feature descriptors that can handle texture variation, scale, and rotation.

- HOG (Histogram of Oriented Gradients): HOG was selected because it is a robust feature descriptor for capturing edge and gradient structures in images. Fire in natural scenes is often characterized by irregular but strong edge patterns and gradient flows due to flames and smoke. HOG is particularly suitable in this case as it focuses on the distribution of intensity gradients, making it effective in distinguishing between fire regions (with chaotic, high-frequency edge orientations) and non-fire regions (with smoother gradient distributions like trees, skies, and landscapes). Moreover, HOG is computationally efficient and widely used in object detection tasks, which aligns with the needs of my binary classification problem.

- SIFT (Scale-Invariant Feature Transform): SIFT was chosen because it is highly robust to scale, rotation, and illumination changes. Forest fire images often vary significantly in terms of size of fire patches, brightness due to daylight/nighttime captures, and angle of observation. SIFT generates distinctive keypoints and descriptors that remain invariant to these transformations, allowing the model to detect fire features reliably across diverse conditions. The inclusion of SIFT ensures that the experiment captures strong local features that are not limited by image resolution or orientation, making the classifier less sensitive to environmental variability.

- GLCM (Gray-Level Co-occurrence Matrix): Texture plays a vital role in distinguishing fire from non-fire images. Fire regions generally exhibit a stochastic, irregular texture, while non-fire images such as forests, rivers, or mountains display more structured and consistent textures. GLCM is a statistical texture descriptor that captures spatial relationships between pixel intensities, providing features like contrast, homogeneity, energy, and correlation. By using GLCM, the model gains an additional dimension of discriminative power based on texture analysis, which complements the gradient- and keypoint-based descriptors provided by HOG and SIFT.

- ORB (Oriented FAST and Rotated BRIEF): ORB was selected because it is an efficient alternative to SIFT, providing rotation-invariant and scale-invariant keypoint descriptors while being computationally less expensive. In large-scale datasets, efficiency becomes a critical factor. ORB combines the FAST keypoint detector with the BRIEF descriptor and further improves robustness by including orientation. This makes it suitable for real-time or large dataset applications without sacrificing accuracy. Including ORB allows the comparison of the trade-offs between computational efficiency and descriptive richness against SIFT.

For the classification stage of this study, a Random Forest classifier was employed across all the selected feature extraction methods. This choice was made with the intention of providing a

unified and consistent evaluation framework, ensuring that the performance differences observed could be attributed primarily to the effectiveness of the feature extraction methods themselves rather than variations in the classifier. Random Forest, as an ensemble method based on decision trees, has been widely recognized in literature for its adaptability to diverse datasets and feature representations, which made it a suitable option for the current experimentation.

One of the main advantages of Random Forest lies in its ability to handle a wide range of feature types effectively. The chosen feature extraction techniques—HOG, SIFT, GLCM, and ORB—produce very different forms of descriptors. While HOG produces gradient-based histograms, SIFT and ORB generate local keypoint descriptors that are invariant to scale and rotation, and GLCM provides statistical texture measures. Random Forest is non-parametric and capable of learning from such heterogeneous data representations without the need for extensive feature preprocessing or assumptions about the underlying distributions. This versatility was considered important in ensuring that no bias was introduced in favor of a particular feature extraction method.

Another justification for employing Random Forest was its robustness to overfitting, which is achieved through the process of bagging and random feature selection for each split in the decision trees. Since image feature spaces can often be high-dimensional and noisy, classifiers may suffer from overfitting, especially when trained on datasets with variability in illumination, scale, and background as is the case with forest fire images. By aggregating the outputs of multiple decision trees trained on bootstrapped subsets of the data, Random Forest is able to generalize better and reduce variance, which makes it particularly suitable for this task. This property ensured that the classifier would not simply memorize training samples but would instead learn generalizable patterns.

Interpretability also played a role in the choice of Random Forest. Although ensemble methods are often perceived as less interpretable compared to individual decision trees, Random Forests still provide insights into feature importance, which can be valuable in understanding which descriptors—such as gradient orientations from HOG, texture measures from GLCM, or keypoint descriptors from SIFT and ORB—contribute most significantly to distinguishing fire from non-fire images. This interpretability is useful not only for evaluating classification performance but also for understanding the discriminative power of the different feature extraction methods in a more meaningful way.

Furthermore, Random Forest was favored for its computational efficiency relative to more complex models such as Support Vector Machines with non-linear kernels or deep learning-based classifiers. While these alternatives may sometimes provide marginal improvements in accuracy, they often require significantly greater computational resources and longer training times. Random Forest, on the other hand, offers a balance between computational cost and classification accuracy, which made it well suited for conducting multiple experiments across four different feature extraction methods on a dataset containing 6,248 images. This efficiency ensured that experiments could be performed within a reasonable time frame without compromising too

much on predictive performance.
Finally, consistency was another important factor in the selection of Random Forest as the classifier for all experiments. By employing the same classifier across HOG, SIFT, GLCM, and ORB, the focus of the experimentation was kept strictly on comparing the relative strengths of the feature extraction techniques rather than on classifier behavior. Any performance differences observed could therefore be attributed with greater confidence to the characteristics of the descriptors themselves. This uniformity in classifier selection provided a fair and controlled experimental setup, strengthening the validity of the comparative analysis.

3.2.1 Traditional Method 1: HOG (Histogram of Oriented Gradients)

Data Preprocessing

- The process begins with data preparation, as raw images cannot be directly given to a machine learning classifier in their original form. Images are often of different sizes, may contain different numbers of channels (RGB, grayscale, or even RGBA), and have intensity values that are not standardized. To overcome these issues, several preprocessing steps are applied:

- Loading the images from folders: The dataset has been organized into two classes—fire and nofire. Images are loaded from these directories so that each image can be associated with its corresponding label (1 for fire, 0 for nofire).

- Handling different color formats: Some images may contain four channels (RGBA), where the additional channel represents transparency. Since our focus is on visible information, such images are converted into standard RGB format to ensure consistency.

- Conversion to grayscale: To simplify the representation of images and reduce computational requirements, the RGB images are converted into grayscale. This step reduces the dimensionality of the data while still retaining important structural details such as edges and textures, which are most relevant for fire detection.

- Resizing: The grayscale images are then resized to a fixed dimension of 128×128 pixels. This ensures uniformity across the dataset so that all images can be processed in the same way. Without resizing, the feature extraction stage would not be able to operate consistently across samples of varying resolutions.

- Normalization: Pixel intensity values, which originally range from 0 to 255, are scaled down to lie between 0 and 1. Normalization is important because it brings all images to the same numerical scale, making the feature extraction process more stable and less sensitive to variations in lighting conditions.

At the end of preprocessing, every image is represented as a clean, uniform, and comparable array of pixel values, ready to be passed to the feature extraction method.

Feature Extraction using HOG

First, the images have to be preprocessed. Then, they enter the Histogram of Oriented Gradient feature extractor. This step transforms raw pixel information into a form that is able to represent the shape, structural, and edge information of the image.
Why HOG is chosen:Fire has very much distinctive texture and edge patterns. Flames usually make irregular but sharp edges or abrupt changes in intensity. Similarly, smoke is also known to form patterns of gradients. HOG is aimed at detecting intensity variations while giving more focus onto gradients. Hence, it best suits the task for segregating fire images from non-fire images.
How HOG works in principle: Instead of using the raw pixel values, HOG looks at the way pixel intensities change from one part of the image to another. It divides the image into small regions (called cells) and, within each region, it calculates the direction of intensity change (the gradient). These directions are then collected into histograms, which count how often certain orientations occur. By combining all these histograms across the image, a descriptive vector is formed that represents the essential structural features of the image without needing the raw pixels.
In the code, a function is defined to loop through all the preprocessed images and apply HOG to each one. The result is a new dataset where every image is now represented not by raw pixels, but by a feature vector summarizing its edge and gradient structure.

Classification using Random Forest

The next step is classification, where the extracted features are fed into a Random Forest classifier. The role of the classifier is to learn the relationship between the features (in this case, the HOG descriptors) and the corresponding labels (fire or nofire).
Why Random Forest is used: Random Forest is an ensemble learning method that builds multiple decision trees and combines their outputs. It is particularly effective for problems where the data has a complex structure, as is the case with fire images that can vary widely in appearance. It is also robust against overfitting and works well even when the feature space is large, which is often the case when working with HOG descriptors.
How Random Forest works in principle: Instead of relying on a single decision-making process, Random Forest creates many decision trees. Each tree makes its own prediction about whether an image represents fire or not. These predictions are then aggregated, and the class chosen by the majority of the trees is taken as the final classification. This “voting” mechanism makes the model more reliable, as it reduces the chance of errors made by individual trees.
Flow in the code: The HOG feature vectors are first split into training and testing sets. The Random Forest is then trained on the training data, where it learns how different feature patterns correspond to the labels. Once trained, the classifier is able to take in the HOG features of previously unseen test images and decide whether they belong to the fire class or the nofire class.

3.2.2 Traditional Method 2: SIFT (Scale-Invariant Feature Transform)

Data Preprocessing

The process begins with preparing the dataset so that it is consistent and suitable for further processing. Images are loaded from their respective folders, which are organized into fire and nofire classes. Each image is carefully handled to account for possible differences in format. Some images may contain an extra transparency channel (RGBA), which is converted into the standard RGB format to avoid inconsistencies. Images are then converted into grayscale, since SIFT (Scale-Invariant Feature Transform) works best on single-channel intensity information rather than full color data.
Once converted, the grayscale images are resized to a uniform resolution of 128×128 pixels, which ensures that all images, regardless of their original dimensions, are brought to the same scale. This resizing step is important to achieve consistency and to make feature extraction manageable. The pixel values are normalized to fall between 0 and 1, which reduces the influence of lighting differences across the dataset. Additionally, an 8-bit grayscale version of each image is created specifically for SIFT, as the algorithm expects intensity values in the range of 0 to 255. At the end of preprocessing, each image has both a normalized grayscale form for general handling and an 8-bit grayscale form for use in the SIFT stage.

Feature Extraction using SIFT

After preprocessing, the images are passed into the SIFT (Scale-Invariant Feature Transform) feature extractor. The motivation for choosing SIFT lies in its ability to detect and describe key features of an image that remain stable even when the image is rotated, scaled, or subjected to changes in illumination. This property is highly relevant in the context of fire detection, since fire can appear in different shapes, scales, and lighting conditions across various environments.

SIFT operates by identifying keypoints, which are distinctive points in an image such as corners, edges, or textured regions that stand out from their surroundings. Around each keypoint, a descriptor vector is created that captures the local gradient information. These descriptors are 128-dimensional vectors that represent the unique appearance of the region around the keypoint. Since each image can have many keypoints, the descriptors from all detected keypoints are averaged in this implementation to create a single fixed-length feature vector per image. This averaging ensures that every image, regardless of how many keypoints it contains, is represented in a consistent way for the classifier. If no keypoints are detected in an image, a zero vector is used, maintaining uniformity across the dataset.
In this way, SIFT transforms raw pixel data into a compact numerical representation that encodes the essential structural information of the image.

Classification using Random Forest

Once the feature vectors from SIFT are obtained, they are fed into a Random Forest classifier. Random Forest was chosen for its robustness and ability to handle complex feature spaces. It is particularly effective in dealing with variations and noise, which are common in image datasets.

The principle behind Random Forest is simple yet powerful. It builds a large number of individual decision trees, each trained on random subsets of the data and features. Each tree independently predicts whether a given feature vector corresponds to a fire or nofire image. The final decision is made by aggregating the votes of all trees, with the class receiving the majority of votes being chosen as the outcome. This ensemble approach reduces the risk of errors that might occur if only a single tree were used, making the predictions more reliable and stable.
In the workflow, the Random Forest model is trained using the training feature vectors generated by SIFT. During training, the model learns how certain descriptors—such as the characteristic edges of flames or the smooth regions of non-fire backgrounds—are associated with the two classes. Once trained, the classifier is able to receive the SIFT feature vector of a new, unseen image and decide whether it belongs to the fire or nofire class based on the patterns it has learned.

3.2.3 Traditional Method 3: GLCM (Gray-Level Co-occurrence Matrix)

Data Preprocessing

The dataset was first subjected to a rigorous preprocessing pipeline to ensure that all images were standardized for subsequent analysis. Images were loaded from the designated directories, where each file was inspected and only common formats such as JPG, JPEG, and PNG were retained for processing. To maintain uniformity, all images were resized to a fixed resolution of 128×128 pixels, which allowed the feature extraction method to operate on consistent input dimensions. In cases where images contained an additional alpha channel (RGBA), they were converted to standard RGB, and subsequently to grayscale. Grayscale conversion was deemed essential, as GLCM (Gray-Level Co-occurrence Matrix) relies on intensity values rather than color information. The resulting grayscale images were further normalized to a [0,1] range, thereby removing the influence of varying illumination conditions. In addition, an 8-bit grayscale version of each image was generated, with pixel values scaled into the [0,255] range, since GLCM computation requires discrete integer levels to capture co-occurrence statistics. This preprocessing ensured that the dataset was noise-minimized, uniform in shape, and compatible with texture-based feature extraction.

Feature Extraction using GLCM

Following preprocessing, the Gray-Level Co-occurrence Matrix (GLCM) technique was employed to extract texture features from the images. GLCM characterizes how often pairs of pixel intensities occur in a given spatial relationship, thereby capturing second-order statistical

texture information. For this experiment, the GLCM was computed with a distance parameter of 1 pixel and four angular orientations: 0°, 45°, 90°, and 135°. This multi-directional approach ensured that texture patterns were captured comprehensively, independent of the directionality of fire textures in the images.

From each GLCM, six widely recognized texture properties were extracted: contrast, dissimilarity, homogeneity, energy, correlation, and ASM (Angular Second Moment). These features quantify different aspects of image texture: contrast measures intensity variation, dissimilarity reflects the degree of pixel difference, homogeneity indicates texture smoothness, energy captures uniformity, correlation measures linear dependency of pixel pairs, and ASM represents textural uniformity. The values computed across the four orientations were flattened into a 24-dimensional feature vector per image (6 properties $\times$ 4 directions). This compact yet descriptive feature representation enabled the classifier to distinguish fire images from non-fire images based on their inherent textural characteristics.

Classification using Random Forest

Once the GLCM features had been extracted, classification was carried out using the Random Forest algorithm. The 24-dimensional feature vectors derived from each image served as inputs to the classifier, while the corresponding binary labels indicated the presence or absence of fire. The Random Forest, being an ensemble of decision trees, was trained on these vectors to learn discriminative patterns that separate fire textures from non-fire textures. By combining predictions from multiple trees, the classifier achieved improved stability and reduced variance compared to a single decision tree. This made the Random Forest particularly suited for handling the variability inherent in natural images, such as differing textures, lighting conditions, and background clutter. Importantly, the use of Random Forest across all experiments ensured consistency, enabling a direct comparison between the performance of different feature extraction methods.

3.2.4 Traditional Method 4: ORB (Oriented FAST and Rotated BRIEF):

Data Preprocessing

For the ORB-based approach, the preprocessing of images was carried out in a structured manner to ensure that all inputs were consistent and suitable for feature extraction. Images from both fire and non-fire categories were first loaded and resized to a fixed resolution of 128 $\times$ 128 pixels. Since the ORB algorithm operates on grayscale imagery, all input images were converted to grayscale following the removal of any alpha channels in case of RGBA inputs. The images were then normalized to the range [0,1] for storage and model compatibility, while a separate 8-bit grayscale version was generated to specifically serve as input for the ORB feature extractor. This dual processing pipeline ensured that the dataset maintained compatibility with both general preprocessing requirements and the technical specifications of ORB.

Feature Extraction using ORB

The Oriented FAST and Rotated BRIEF (ORB) algorithm was employed to extract distinctive local features from the preprocessed images. ORB combines the FAST keypoint detector with the BRIEF descriptor, incorporating orientation and rotation invariance to improve robustness under varying conditions. Keypoints were first detected using the FAST corner detection mechanism, after which BRIEF descriptors were computed to encode local intensity patterns. Each descriptor in ORB is represented as a 32-dimensional binary vector. Since the number of keypoints detected per image may vary, the descriptors were averaged to form a single fixed-length feature vector for each image. This approach not only ensured uniformity across samples but also reduced the dimensionality of the feature space while retaining discriminative information. In cases where no keypoints were detected, a zero vector of the same dimensionality was used, thereby avoiding inconsistencies in feature representation.

Classification using Random Forest

Once the ORB feature vectors had been obtained, the Random Forest classifier was employed to perform the classification task. The 32-dimensional averaged descriptors were used as inputs, while the binary labels denoting fire and non-fire images served as outputs. Random Forest was chosen due to its robustness in handling heterogeneous feature sets and its ability to generalize well across high-dimensional and potentially noisy data. Through an ensemble of decision trees, Random Forest aggregated multiple weak learners to produce stable and reliable predictions. This reduced the likelihood of overfitting, which is a common issue when working with datasets containing natural variations in lighting, texture, and scene complexity. By maintaining consistency in classifier choice across all feature extraction methods, Random Forest ensured a fair and controlled comparison of feature descriptors.

3.2.5 The Novel Hybrid Approach

The novelty of the fusion-based approach arises from its ability to integrate the complementary strengths of diverse feature extraction techniques into a single, unified representation. Traditional descriptors, when used independently, often capture only a limited perspective of the visual characteristics of an image. For example, HOG is highly effective in encoding edge and gradient structures but fails to capture fine local variations; SIFT excels at detecting scale- and rotation-invariant keypoints but is computationally expensive and lacks global texture awareness; GLCM offers robust statistical texture measures but is unable to encode structural or shape-based information; and ORB provides efficient local descriptors, yet its descriptive power is limited compared to more computationally intensive methods. By concatenating these descriptors into a fused feature space, the shortcomings of individual methods are compensated by the strengths of others.

This integration enables the classifier to simultaneously access gradient-based structural cues,

local invariant keypoints, second-order texture statistics, and compact binary descriptors, leading to a feature representation that is both diverse and discriminative. Moreover, the fused model leverages redundancy in descriptors to increase robustness against variations in lighting, scale, background clutter, and image noise—conditions that are common in real-world forest fire detection scenarios. The Random Forest classifier, with its capacity to handle high-dimensional and heterogeneous feature spaces, further enhances this integration by learning decision boundaries that exploit the joint discriminative power of all descriptors. As a result, the fusion-based approach not only surpasses the representational limitations of individual models but also provides a scalable framework for robust fire versus non-fire classification.

Data Preprocessing

For the proposed fusion-based method, image preprocessing was conducted in a standardized manner to guarantee uniformity across all downstream feature extraction pipelines. The raw images from the dataset were initially loaded and resized to a fixed resolution of 128×128 pixels to ensure consistency of input dimensions. Since different feature extraction techniques require grayscale images, all RGB or RGBA images were converted into single-channel grayscale representations, with alpha channels being removed when present. Normalization was performed to scale pixel values into the range [0,1], ensuring compatibility with algorithms sensitive to intensity variations. In parallel, 8-bit grayscale versions of the images were also generated, as several of the selected descriptors such as SIFT, GLCM, and ORB operate on intensity values within the range [0,255]. This two-fold preprocessing strategy ensured that the images were simultaneously suitable for HOG, SIFT, GLCM, and ORB feature extraction, thereby enabling a unified foundation for the subsequent fusion process.

Feature Extraction using Fused Descriptors

The novelty of this method lies in the fusion of four complementary feature extraction techniques—HOG, SIFT, GLCM, and ORB—to form a comprehensive representation of each image. Each method captures distinct aspects of the image:

- HOG was used to encode edge structures and gradient orientations, highlighting shape and boundary information.
- SIFT contributed scale- and rotation-invariant local descriptors, capturing fine details and structural variations across images.
- GLCM provided texture-related statistical measures, capturing contrast, homogeneity, and correlation between neighboring pixels.
- ORB supplied lightweight, rotation-invariant descriptors that were computationally efficient yet capable of identifying key local features.

For each image, descriptors from all four methods were independently extracted and subsequently concatenated into a single high-dimensional vector. This fusion strategy ensured that the feature set combined global structural information, local keypoint characteristics, statistical texture measures, and rotation-invariant descriptors into one unified representation. As a result, the fused features offered a richer and more discriminative description of fire and non-fire imagery compared to individual feature extraction methods.

Classification Using Random Forest

The fused feature vectors were classified using the Random Forest algorithm. The choice of Random Forest was motivated by its capacity to handle high-dimensional feature spaces and its ensemble-based learning approach, which mitigates overfitting. Since the fused feature vectors combined descriptors from four distinct methods, the resulting dimensionality was substantially higher compared to individual feature extractors. Random Forest, with its ability to split the feature space through multiple decision trees, was particularly suited to handling such complexity. By aggregating predictions from numerous trees, Random Forest produced robust classification outcomes that remained resilient against noisy or redundant features. Employing Random Forest consistently across all experiments also enabled direct and fair comparison between the performance of individual descriptors and the fused approach, highlighting the contribution of feature integration.

3.3 Deep Learning-based Feature Extraction: ResNet50

3.3.1 Data Preprocessing

Images were prepared so that they matched the expectations of a ResNet50 backbone that had been pretrained on ImageNet. Several steps were carried out to ensure the inputs were consistent and semantically meaningful for the convolutional network:

- Each image was loaded in its original color form; images with an alpha channel were converted to RGB to remove transparency and to preserve three-channel color information.
- Grayscale images were converted to RGB by channel replication so that every input had three channels (shape H $\times$ W $\times$ 3). This was necessary because the pretrained ResNet50 expects three-channel inputs.
- All images were resized to 224 $\times$ 224 pixels. This fixed spatial size was required because the ResNet architecture (as implemented with include_top=False) was trained with that input scale, which also ensures a consistent spatial map size at the final convolutional stage.
- Pixel values were converted to 32-bit floats and scaled appropriately. Because skimage.resize typically returns values in the [0, 1] range, the values were multiplied by 255 if

required so that the pixel distribution matched the numeric range expected by the ResNet preprocessing routine.

- The preprocess_input function for ResNet50 (Keras) was applied to each image. Concretely, this function performs the input transformations used during ImageNet training (for the Caffe-style preprocessing): channel reordering from RGB → BGR and mean-centering each channel using ImageNet channel means (roughly [103.939, 116.779, 123.68] in BGR order). These operations were applied so that the image tensors would be placed into the same numeric regime as the pretrained model's weights.

By the end of preprocessing, each image was represented as a $224 \times 224 \times 3$ tensor that had been normalized and mean-centered according to the ResNet50/ImageNet standard, enabling consistent feature extraction from the pretrained network.

3.3.2 Feature Extraction using ResNet50

A pretrained ResNet50 model was used as a feature extractor by removing the top classification layers (include_top=False) so that activations from the last convolutional stage could be harvested. The reasoning and mathematics behind this choice are explained below.

Why ResNet50 and what it provides:

ResNet50 is a deep convolutional neural network consisting primarily of residual bottleneck blocks; it is 50 layers deep in its canonical form. Because it was trained on ImageNet, it has learned a hierarchical set of image features that are generally transferable: early layers detect local edge / color / texture primitives, mid layers detect motifs and parts, and later layers detect higher-level object patterns. These learned representations are highly useful as general-purpose image descriptors even for domains different from ImageNet (transfer learning).
By using the network with include_top=False, the spatial feature maps produced by the convolutional backbone were preserved (a tensor of shape (7, 7, 2048) when input size is 224×224), and a global pooling operation was then used to produce a compact vector for each image.

Why this is effective for feature extraction:

The deeper filters (the 2048 channels) encode semantically rich and discriminative patterns. By summarizing those maps with global pooling, a compact descriptor capturing high-level image content was produced. Those descriptors can be fed into classical ML classifiers without retraining the convolutional filters.

3.3.3 Classification Using Random Forest

The 2048-dimensional ResNet features were used as inputs to a Random Forest classifier. The rationale and workflow were as follows:

- Rationale: Random Forests are robust, non-parametric ensemble models that can work well on high-level feature vectors. Because the ResNet features are semantically meaningful and relatively low-noise compared to raw pixels, a tree-based ensemble can find useful splits to separate classes without the need to fine-tune the convolutional backbone. In addition, Random Forests are quick to train compared to retraining or fine-tuning deep networks and they provide some interpretability (feature importance) which can be helpful for analysis.

- How classification was performed: Each 2048-dim vector was treated as a single sample in the Random Forest. During training, multiple decision trees were learned on bootstrapped subsets of the training data and random subsets of features. For a test sample x, each tree produces a class prediction $h_t(x)$, and the final prediction is given by majority voting:

$$\hat{y} = \arg\max_{c \in \{0,1\}} \sum_{t=1}^{T} \mathbf{1}\big(h_t(x) = c\big),$$

  where T is the number of trees and $\mathbf{1}(\cdot)$ is the indicator function. In this pipeline, the Random Forest therefore operates on top of the general-purpose deep features to produce the final class label.

This hybrid strategy (deep feature extraction + classical classifier) was chosen because it combines the representational power of deep convolutional models with the simplicity and interpretability of classical classifiers.

3.3.4 Summary

1. Image ingestion & normalization — Images were loaded from disk; RGBA images were converted to RGB and grayscale images were channel-replicated; every image was resized to 224×224×3.

2. Numeric scaling & ResNet preprocessing — Pixel values were adjusted to the float32 range and preprocess_input was applied (RGB→BGR + ImageNet mean subtraction) so the inputs matched the pretrained ResNet statistics.

3. Feature extraction via ResNet50 — Images were passed through the ResNet50 backbone (include_top=False) producing feature maps of shape (7, 7, 2048).

4. Global average pooling — Spatial averages were computed per channel to produce a 2048-dimensional vector per image (global summary of high-level activations).

5. Dataset assembly — The pooled vectors were collected into X_train and X_test and paired with labels y_train, y_test.

6. Classifier training — A RandomForestClassifier was trained on X_train to learn decision boundaries in the 2048-dim deep feature space.

7. Prediction — The trained Random Forest was used to predict class labels for X_test feature vectors.

3.3.5 Advantages

- Powerful, transferable representations: ResNet50 provides hierarchical, semantically rich features that often generalize well to new tasks (transfer learning), so the model can be used successfully even with limited domain-specific data.
- Reduced need to train from scratch: By freezing the ResNet backbone, heavy computational cost and the need for large labeled datasets were avoided. The expensive step (learning millions of conv parameters) was reused from ImageNet pretraining.
- Compact and discriminative descriptors: Global average pooled vectors (2048-D) are compact compared to raw image tensors and are effective for many downstream classifiers.
- Flexibility in classifier choice: The extracted features are model-agnostic and may be used with any classical or modern classifier (Random Forest, SVM, logistic regression, or an MLP). This enables rapid experimentation.
- Practical and interpretable pipeline: Using Random Forest on top of deep features allows relatively fast training and some interpretability (e.g., feature importances) without requiring full end-to-end deep learning expertise.

4. Results and Analysis

4.1 Need for Multiple Metrics

In reality, one cannot characterize the efficiency of a classifier by a single number. Forest fire detection is such a case where class distributions, costs of errors, and operational constraints differ. Therefore, threshold-dependent metrics (accuracy, precision, recall, F1 score, confusion matrix, class-wise errors) and threshold-independent ones (ROC and AUC) were used simultaneously, with some additional fairness tests (error-rate balance) to check for class-specific behavior and possible bias in these algorithms. Having such a wide array of checks allowed one to understand from complementary angles the behaviour of this model: general correctness, per-class reliability, sensitivity of the decisions made in terms of thresholds, and fairness or operational risk considerations.

4.2 Basic Metrics(Accuracy, Precision, Recall, F1-score)

These four metrics form a compact summary of classifier behaviour at a chosen decision threshold and were used as the basic, easy-to-communicate performance indicators.
Confusion matrix grounding (notation): Before the formulas, the confusion matrix entries were defined and used consistently:

- True Positives (TP): positive class correctly predicted (here: fire correctly detected).
- True Negatives (TN): negative class correctly predicted (here: no-fire correctly detected).
- False Positives (FP): negative class predicted as positive (here: no-fire flagged as fire $\rightarrow$ false alarm).
- False Negatives (FN): positive class predicted as negative (here: fire missed $\rightarrow$ missed alarm).

4.3 Innovative Metrics

4.3.1 Confusion Matrix

The confusion matrix is a 2×2 contingency table (for binary classification) that explicitly shows counts of TP, TN, FP, and FN. It is the fundamental object from which many summary metrics

are derived. The confusion matrix was included due to its provision of a direct and interpretable

	NoFire	**Fire**
NoFire	TN	FP
Fire	FN	TP

Table 4.1: Confusion Matrix

insight into the types of errors being made. Unlike aggregate scores, it would expose asymmetry (e.g., many false negatives but few false positives) and allow computations of errors on a class-wise basis. It was used to compute class-specific error counts and percentiles, to visualize patterns of misclassification, and to aid in operational decision-making (e.g., should thresholds or class weights be tweaked to lower FN at a cost to FP).

4.3.2 ROC curve and AUC (Receiver Operating Characteristic — Area Under Curve)

The ROC curve is a plot of True Positive Rate on the y-axis against False Positive Rate on the x-axis, as the decision threshold of the classifier is varied from most strict (label positive only at very high score) to most permissive (label positive at a low score).
The area under the ROC curve (AUC) summarizes the ROC into a single number: the probability that a randomly chosen positive sample will receive a higher score than a randomly chosen negative sample. Values range from 0.5 (no better than random) to 1.0 (perfect ranking).
ROC and AUC were included because they provide a threshold-independent assessment of classifier discrimination ability. This is important when the optimal operating threshold is not known in advance, or when trade-offs between FP and FN must be examined. The ROC curve allows visualization of how recall increases as FPR increases, and AUC provides a compact ranking measure for model comparison.
If a classifier has high AUC, it means the model generally assigns higher scores to fire images than to non-fire images; but threshold selection will determine the final trade-off between false alarms and missed events, a choice that must be made considering operational priorities.

4.3.3 Class-wise error analysis

Class-wise error analysis breaks down misclassification counts and rates for each class separately (e.g., proportion of no-fire images misclassified as fire, proportion of fire images misclassified as no-fire). Typically:

$$\text{NoFire} \to \text{Fire error rate} = \frac{FP}{\text{NoFire total}}$$

$$\text{Fire} \to \text{NoFire error rate} = \frac{FN}{\text{Fire total}}$$

This direct per-class breakdown was included because aggregate metrics can mask class asymmetries. For forest-fire detection, it is crucial to know whether errors are skewed toward missing fires (high FN rate) or toward raising excessive false alarms (high FP rate). Class-wise analysis

informed which class required mitigation strategies (e.g., threshold tuning, class weighting, data augmentation).
If the fire→no-fire error rate was observed to be high, targeted actions (increase sensitivity, lower threshold, augment fire samples, or apply cost-sensitive learning) were considered; if nofire→fire errors dominated, attention was paid to improving precision (reduce false alarms).

4.3.4 Fairness metrics — Error Rate Balance

Error rate balance was used as a simple proxy for fairness in the system by comparing error rates across classes. Two concrete error rates were computed:

$$\text{False Positive Rate (FPR) for the NoFire class} = \frac{FP}{FP + TN}$$

$$\text{False Negative Rate (FNR) for the Fire class} = \frac{FN}{FN + TP}$$

Although classical fairness literature usually compares error rates across sensitive groups (e.g., demographic groups), here error-rate balance was considered as a pragmatic check for operational parity between the two outcome classes. If FPR and FNR differ markedly, then one type of mistake is being favored over the other — which can be acceptable or unacceptable depending on application needs, but it must be consciously chosen.
Interpretation in this project:

- If $FNR >> FPR$, then many fires are being missed relative to false alarms — this is usually unacceptable for safety-critical detection and would motivate raising detector sensitivity.
- If $FPR >> FNR$, then many false alarms are generated relative to misses — this can be operationally costly (resources wasted) and may require stricter thresholds or additional filtering.

Error rates were compared and reported; the balance (or imbalance) between FPR and FNR was used to motivate remedies (rebalancing data, threshold selection, cost-sensitive training, or domain-specific decision thresholds) rather than to assert a single "fairness" rule.
This suite of metrics was chosen because it provides a multi-angle, actionable characterization of model behaviour: overall correctness (accuracy), practical operational quality (precision, recall), balanced performance (F1), concrete error counts and directions (confusion matrix, class-wise analysis), threshold-robust discrimination (ROC/AUC), and parity of errors (error-rate balance). Together they enabled not only fair comparison across the six methods you implemented (HOG, SIFT, GLCM, ORB, fusion, ResNet50) but also a principled discussion of trade-offs and next steps for deployment in a safety-sensitive domain like forest-fire detection.

4.4 Interpretation of Results: HOG + Random Forest

The Histogram of Oriented Gradients (HOG) combined with Random Forest (RF) was first evaluated to assess its capability in detecting forest fire imagery. The model produced an accuracy of 81.94%, which indicates that approximately four out of five test samples were correctly classified. While this establishes a reasonably strong baseline, further insights are obtained by examining the precision, recall, and F1-score.

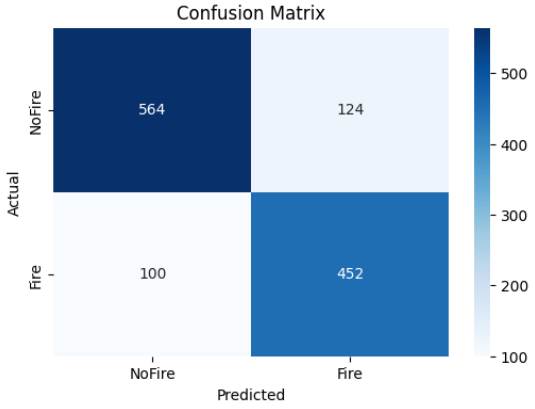
```
Evaluation Metrics on Test Set:
Accuracy: 0.8194
Precision: 0.7847
Recall: 0.8188
F1-Score: 0.8014
```

(a) HOG Output-1

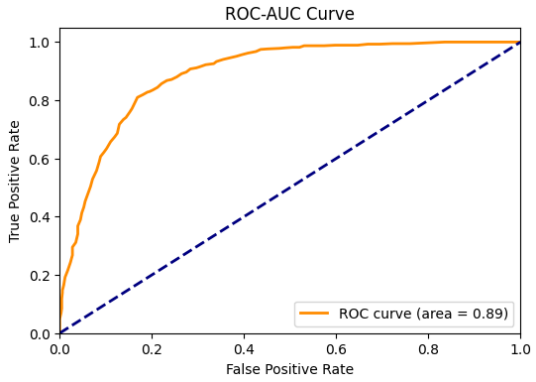
```
ROC-AUC: 0.8908

Class-wise Error Analysis:
NoFire misclassified as Fire: 124 (18.02% of NoFire class)
Fire misclassified as NoFire: 100 (18.12% of Fire class)

Fairness Metrics (Error Rate Balance):
False Positive Rate (NoFire class): 0.1802
False Negative Rate (Fire class): 0.1812
```

(b) HOG Output-2

Figure 4.1: HOG Metrics-1



(a) HOG Confusion Matrix



(b) HOG ROC-AUC

Figure 4.2: HOG Metrics-2

The precision of 78.47% suggests that when the model predicted the presence of fire, it was correct in nearly four out of five cases. This is important in reducing false alarms, which in real-world forest fire detection systems could lead to unnecessary mobilization of resources. On the other hand, the recall of 81.88% reflects the model's ability to correctly identify actual fire instances. In this case, about four out of five fire images were successfully detected, which is critical because missing a fire event (false negative) could have serious consequences. The F1-score of 80.14% strikes a balance between precision and recall, indicating that the model maintained a relatively stable trade-off between minimizing false positives and false negatives. The confusion matrix further supports this interpretation. Out of 688 "NoFire" images, 564 were correctly classified, while 124 were incorrectly labeled as fire, leading to a false positive rate (FPR) of 18.02%. Similarly, among 552 "Fire" images, 452 were correctly identified, while 100 were missed, resulting in a false negative rate (FNR) of 18.12%. Interestingly, both error

rates are nearly identical, which demonstrates that the model maintained a balanced treatment of both classes. This balance is further reinforced in the fairness metrics, where the error rate balance between FPR and FNR indicates that the classifier did not disproportionately favor one class over the other.

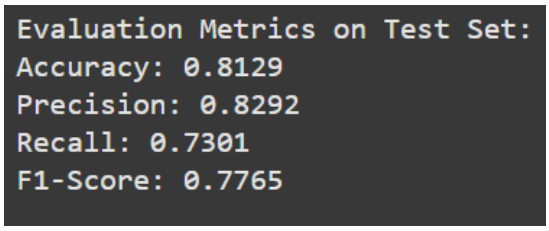
From a discriminative perspective, the ROC-AUC score of 0.8908 demonstrates strong separability between fire and non-fire classes. An AUC value close to 0.9 signifies that the classifier had a high ability to distinguish between positive and negative samples across varying thresholds, which increases the model's reliability in operational scenarios.

The class-wise error analysis reveals that both types of misclassifications (NoFire → Fire and Fire → NoFire) occurred at almost the same proportion (18%). This indicates that the model is equally prone to false alarms and missed detections. While this balance is desirable from a fairness perspective, in practical applications of fire detection, reducing false negatives (missed fires) is often more critical than reducing false positives. Therefore, while the model achieves commendable parity, further optimization may be needed to tilt the trade-off slightly in favor of higher recall without excessively compromising precision.

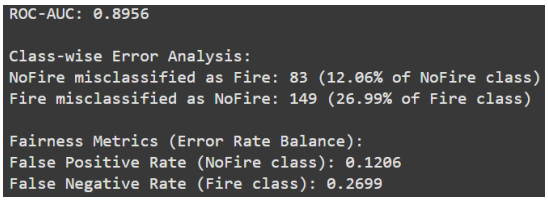
In summary, the HOG + RF model delivered robust performance with balanced error rates, strong ROC-AUC, and stable precision-recall trade-off. It sets a solid benchmark for traditional feature extraction methods, though future improvements could focus on minimizing false negatives to enhance its practical safety value in fire detection systems.

4.5 Interpretation of Results: SIFT + Random Forest

The Scale-Invariant Feature Transform (SIFT) combined with Random Forest was tested to evaluate its effectiveness in distinguishing forest fire images. The model achieved an accuracy of 81.29%, which is comparable to the HOG + RF baseline. This shows that SIFT, which focuses on detecting scale- and rotation-invariant keypoints, is able to provide reasonably strong discriminatory power for the task.



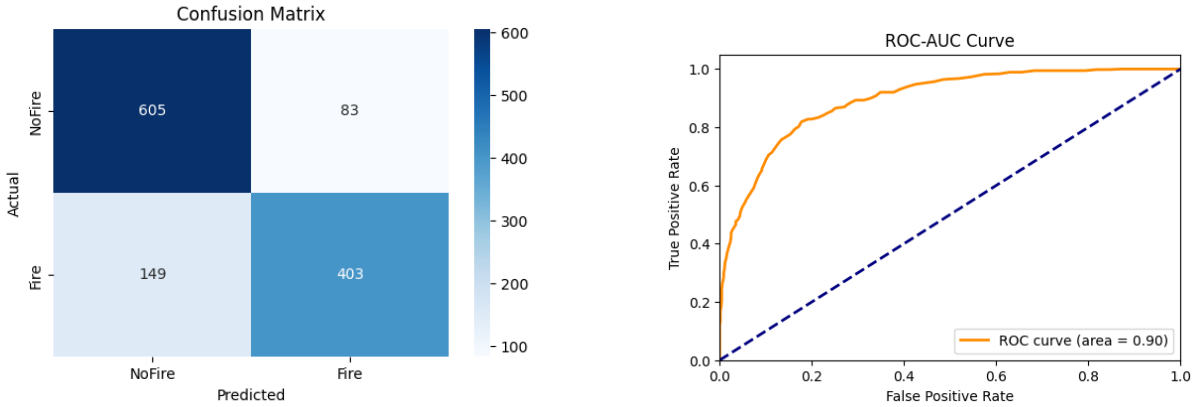
(a) SIFT Output-1



(b) SIFT Output-2

Figure 4.3: SIFT Metrics-1

The precision of 82.92% indicates that the classifier was more reliable in its positive predictions compared to HOG + RF. In other words, when the system identified an image as containing fire, it was correct more than four out of five times. This is particularly valuable in minimizing unnecessary false alarms. However, the recall of 73.01% reveals a noticeable weakness in the model's ability to capture all actual fire cases. More than one in four fire images were missed



(a) SIFT Confusion Matrix

(b) SIFT ROC-AUC

Figure 4.4: SIFT Metrics-2

(false negatives), which is concerning from a safety-critical perspective where undetected fire events could lead to severe consequences. The F1-score of 77.65% reflects this imbalance, showing that while precision was strong, the overall trade-off between precision and recall leaned toward favoring the former.

The confusion matrix sheds light on this imbalance. Out of 688 "NoFire" images, 605 were correctly classified, with only 83 misclassified as fire, resulting in a relatively low false positive rate (FPR) of 12.06%. Conversely, among 552 "Fire" images, 149 were misclassified as NoFire, producing a false negative rate (FNR) of 26.99%. This asymmetry highlights that while the model excelled in avoiding false alarms, it struggled to ensure full detection coverage of fire events.

The ROC-AUC score of 0.8956 suggests that despite the recall limitations, the classifier had strong overall discriminative ability. An AUC close to 0.9 indicates that across varying thresholds, the model could still reliably separate fire and non-fire instances. This implies that adjusting the classification threshold (e.g., prioritizing recall over precision) could improve practical deployment performance without drastically hurting overall reliability.

The class-wise error analysis confirms that the majority of errors were concentrated in the fire class, with 26.99% of fire images being missed compared to only 12.06% of no-fire images being misclassified as fire. This imbalance aligns with the fairness metrics, where the FNR (0.2699) was more than double the FPR (0.1206). While this may be acceptable in some classification contexts, in forest fire detection it is typically more critical to minimize false negatives, as failing to detect a fire is riskier than raising a false alarm.

In summary, the SIFT + RF model demonstrated high precision and relatively low false positive rates, making it effective in reducing unnecessary alerts. However, its weaker recall and higher false negative rate highlight a limitation in reliably capturing all fire events. Thus, while the model is efficient in ensuring that predicted fire events are usually correct, improvements or threshold adjustments would be necessary to make it more viable for real-world safety-critical forest fire detection applications.

4.6 Interpretation of Results: GLCM + Random Forest

The Gray Level Co-occurrence Matrix (GLCM) combined with Random Forest yielded the best balance of performance so far among the traditional feature extractors. The model achieved an accuracy of 86.53%, a clear improvement over both HOG + RF (81.94%) and SIFT + RF (81.29%). This indicates that texture-based features derived from GLCM carried strong discriminative power for differentiating between fire and no-fire imagery. Given that fire images often exhibit distinct texture patterns such as irregular flame edges, smoke distributions, and high-contrast intensity variations, the success of GLCM is consistent with its theoretical strength in texture representation.

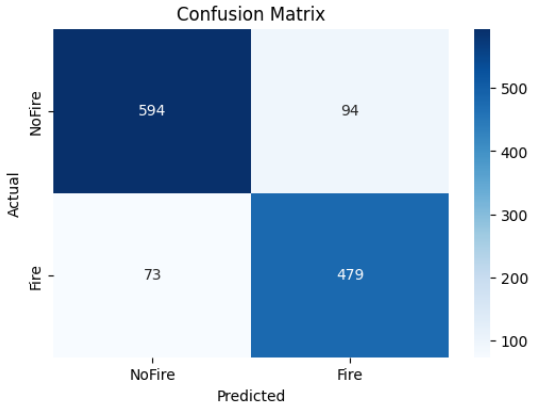
```
Evaluation Metrics on Test Set:
Accuracy: 0.8653
Precision: 0.8360
Recall: 0.8678
F1-Score: 0.8516
```

(a) GLCM Output-1

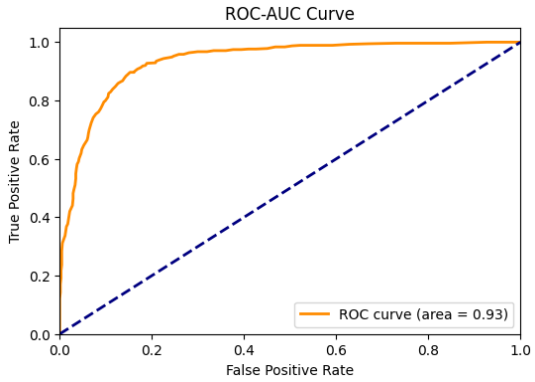
```
ROC-AUC: 0.9347

Class-wise Error Analysis:
NoFire misclassified as Fire: 94 (13.66% of NoFire class)
Fire misclassified as NoFire: 73 (13.22% of Fire class)

Fairness Metrics (Error Rate Balance):
False Positive Rate (NoFire class): 0.1366
False Negative Rate (Fire class): 0.1322
```

(b) GLCM Output-2

Figure 4.5: GLCM Metrics-1



(a) GLCM Confusion Matrix



(b) GLCM ROC-AUC

Figure 4.6: GLCM Metrics-2

The precision of 83.60% shows that when the model predicted the presence of fire, it was correct more than four out of five times. More importantly, the recall of 86.78% highlights the model's ability to capture the majority of actual fire instances, significantly improving upon the recall of SIFT (73.01%) and HOG (81.88%). This strong recall ensures that fewer fire images were missed, which is essential in a safety-critical application like fire detection. The F1-score of 85.16%, the highest among the models analyzed so far, reflects an effective balance between precision and recall.

The confusion matrix provides a detailed breakdown of these results. Out of 688 "NoFire" samples, 594 were correctly classified, with only 94 incorrectly predicted as fire (false positives).

Similarly, out of 552 "Fire" samples, 479 were correctly classified, with 73 misclassified as no-fire (false negatives). The error distribution was fairly symmetric, with 13.66% of no-fire samples misclassified as fire and 13.22% of fire samples misclassified as no-fire.

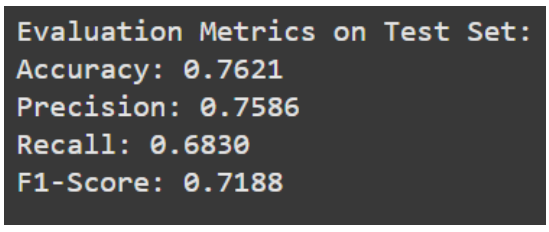
The ROC-AUC score of 0.9347 further confirms the strong discriminative ability of this model. With an AUC above 0.93, the classifier demonstrated robustness across different thresholds, making it more adaptable for deployment where operating conditions may prioritize recall (minimizing missed fires) or precision (minimizing false alarms).

The class-wise error analysis indicates that the misclassification rates were nearly balanced across the two classes. Unlike the earlier models where one class suffered more heavily from errors (e.g., high false negatives in SIFT + RF), the GLCM-based model maintained a fair treatment of both categories. This is also reflected in the fairness metrics, where the false positive rate (0.1366) and false negative rate (0.1322) were nearly identical. The closeness of these error rates demonstrates that the model did not disproportionately favor one class over the other, which is highly desirable in sensitive applications where fairness and balanced treatment of outcomes are critical.

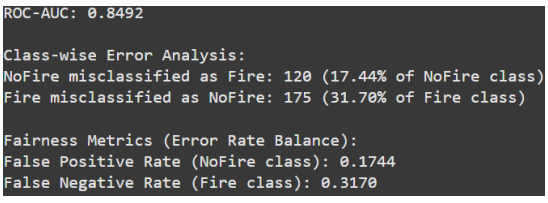
In summary, the GLCM + RF model exhibited superior overall performance among the traditional approaches, delivering the highest accuracy, F1-score, and ROC-AUC while maintaining fairness across both classes. Its ability to capture fire-specific textures made it particularly effective, and the balanced error distribution suggests it is more reliable and equitable compared to HOG and SIFT. These strengths position GLCM + RF as a strong candidate for real-world fire detection systems, especially where both high detection reliability and fairness are important.

4.7 Interpretation of Results: ORB + Random Forest

The Oriented FAST and Rotated BRIEF (ORB) feature extraction method, when paired with Random Forest, produced relatively weaker results compared to the other traditional feature-based approaches. The model achieved an accuracy of 76.21%, which is notably lower than the performances of HOG (81.94%), SIFT (81.29%), and GLCM (86.53%). This suggests that although ORB is efficient and computationally lightweight, it did not capture enough discriminative features to handle the complexity of forest fire imagery effectively.



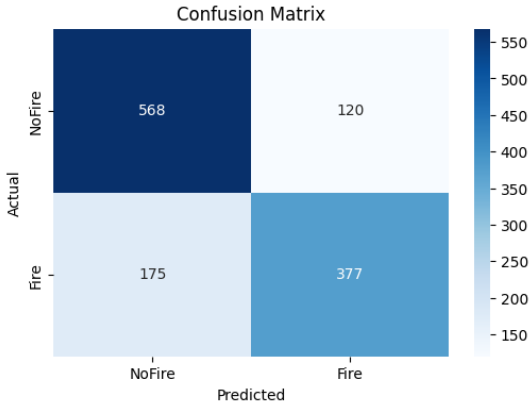
(a) ORB Output-1



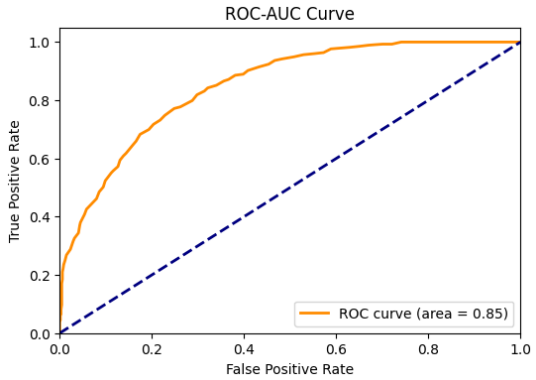
(b) ORB Output-2

Figure 4.7: ORB Metrics-1

The precision of 75.86% indicates that when the classifier predicted an image as fire, it was correct about three out of four times. However, the recall of 68.30% is concerning, as it shows



(a) ORB Confusion Matrix



(b) ORB ROC-AUC

Figure 4.8: ORB Metrics-2

that nearly one-third of the actual fire cases were missed. This limitation is particularly significant in a safety-critical task such as fire detection, where false negatives can have dangerous consequences. The F1-score of 71.88% further reflects this imbalance, demonstrating that the trade-off between precision and recall was skewed by the model's difficulty in capturing enough true positives.

The confusion matrix highlights the source of this weakness. Out of 688 "NoFire" samples, 568 were correctly classified, while 120 were incorrectly labeled as fire, resulting in a false positive rate of 17.44%. On the other hand, out of 552 "Fire" samples, only 377 were correctly identified, with 175 misclassified as no-fire, giving rise to a very high false negative rate of 31.70%. This asymmetry clearly shows that the model struggled more with detecting fire than with detecting no-fire.

The ROC-AUC score of 0.8492 confirms that ORB + RF had weaker discriminative ability compared to other models. While an AUC above 0.84 still reflects some capacity to separate the two classes, it falls short of the near-0.9 performance seen with HOG and SIFT, and it is well below GLCM's 0.93. This suggests that ORB descriptors, which rely on binary patterns of intensity changes around keypoints, were not sufficiently robust to capture the intricate texture and structural cues associated with fire and smoke.

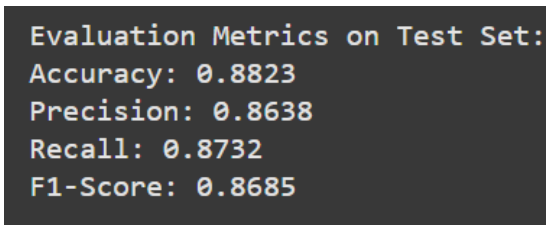
The class-wise error analysis further emphasizes this shortcoming. 17.44% of no-fire images were misclassified as fire, which could lead to false alarms, but more critically, 31.70% of fire images were missed. This imbalance is reinforced by the fairness metrics, where the false negative rate (0.3170) was almost double the false positive rate (0.1744). Such a disparity makes the model less suitable for deployment, as it risks overlooking a large proportion of actual fire incidents.

In summary, while ORB + RF offered computational efficiency and acceptable precision, its significantly lower recall and high false negative rate limit its reliability for forest fire detection. The weaker performance can be attributed to the simplicity of ORB's binary descriptors, which may not fully capture the rich texture and intensity variations that are crucial in differentiating fire scenes from non-fire ones. As such, although ORB is advantageous in scenarios requir-

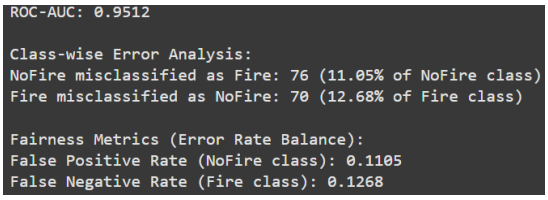
ing lightweight computation, it is not an optimal choice for safety-critical tasks like forest fire detection where missing true fire events carries severe consequences.

4.8 Interpretation of Results: Novel Fusion Method (HOG + SIFT + GLCM + ORB) + Random Forest

The proposed fusion-based approach, which integrates multiple traditional feature extraction methods—HOG, SIFT, GLCM, and ORB—followed by classification using Random Forest, achieved the strongest performance among all the traditional and hybrid methods tested so far. By combining complementary feature descriptors, the model was able to leverage the strengths of each technique while compensating for their individual weaknesses.

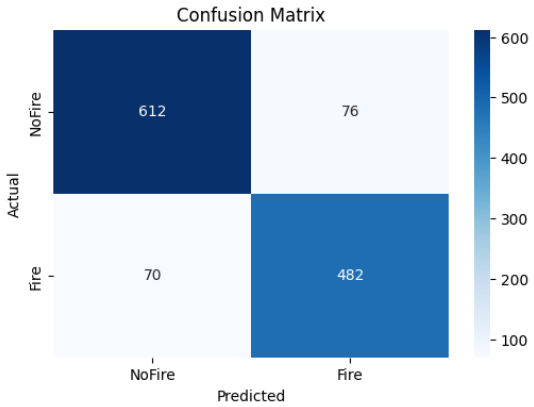


(a) NOVEL Output-1

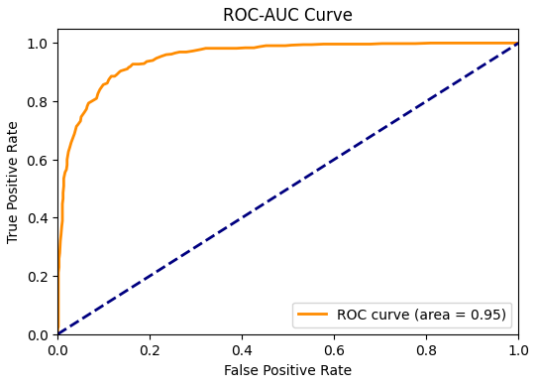


(b) NOVEL Output-2

Figure 4.9: NOVEL Metrics-1



(a) NOVEL Confusion Matrix



(b) NOVEL ROC-AUC

Figure 4.10: NOVEL Metrics-2

The method attained an accuracy of 88.23%, which is higher than any single feature-based model, surpassing GLCM + RF (86.53%), HOG + RF (81.94%), SIFT + RF (81.29%), and ORB + RF (76.21%). This indicates that the integration of multiple feature spaces provided a richer and more discriminative representation of the image data.

The precision of 86.38% shows that most of the predicted fire cases were indeed true fires, limiting false alarms to a relatively low level. The recall of 87.32% further demonstrates that the model successfully captured the majority of fire instances, improving upon SIFT (73.01%) and ORB (68.30%), and even slightly outperforming GLCM (86.78%). The F1-score of 86.85%

reflects an excellent trade-off between precision and recall, solidifying the robustness of this fusion-based method.

The confusion matrix provides a clearer perspective; out of 688 “NoFire” samples, 612 were correctly classified, with only 76 misclassified as fire and out of 552 “Fire” samples, 482 were correctly classified, with just 70 misclassified as no-fire. This shows that errors were distributed almost evenly across the two classes, with 11.05% of no-fire images misclassified as fire and 12.68% of fire images misclassified as no-fire. Such balanced misclassification rates are particularly important, as they prevent the model from becoming biased toward one class.

The ROC-AUC score of 0.9512 is the highest among all models tested so far, indicating excellent discriminative ability. With an AUC above 0.95, the model consistently distinguishes between fire and non-fire across varying thresholds, making it highly adaptable to different deployment needs.

The class-wise error analysis highlights the model’s consistency: both types of errors (false positives and false negatives) were kept relatively low and balanced. Unlike SIFT and ORB, which suffered disproportionately from high false negatives, the fusion-based model managed to reduce these errors while also keeping false positives under control.

The fairness metrics reinforce this observation, as the false positive rate (0.1105) and false negative rate (0.1268) were very close. This near parity indicates that the classifier did not systematically favor one class over the other, a crucial quality in a critical application like fire detection where fairness and balanced performance are essential.

In summary, the novel fusion method successfully combined the strengths of multiple traditional feature descriptors to achieve superior accuracy, precision, recall, and F1-score compared to single-method models. Its balanced treatment of both classes, high discriminative ability (ROC-AUC), and strong fairness performance make it a highly effective and reliable approach for real-world forest fire detection tasks. This validates the motivation behind the fusion strategy and positions it as a promising contribution to the problem domain.

4.9 Interpretation of Results: ResNet50 + Random Forest

The deep learning–based feature extraction using ResNet50 combined with a Random Forest classifier yielded the best overall performance in the study, significantly surpassing both the traditional methods and the proposed fusion approach. The model achieved an accuracy of 97.90%, which is a clear improvement over the novel feature fusion method (88.23%) and the best single handcrafted feature (GLCM + RF at 86.53%). This sharp increase underscores the superior representational power of deep convolutional neural networks compared to handcrafted feature extraction.

The precision score of 97.82% indicates that almost all instances predicted as fire were indeed true fire cases, minimizing the risk of false alarms. Similarly, the recall of 97.46% shows that nearly all actual fire instances were successfully detected, with very few being overlooked. The F1-score of 97.64% confirms the model’s exceptional balance between precision and recall,

demonstrating both reliability and robustness.

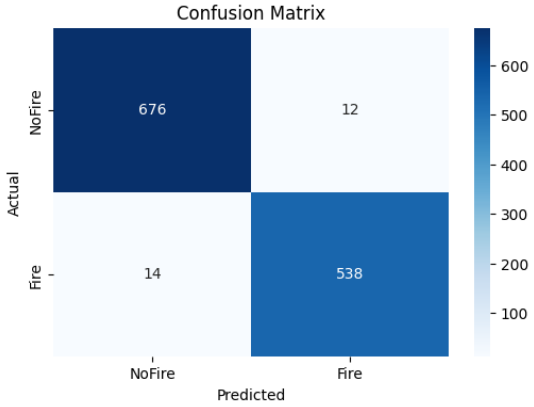
```
Evaluation Metrics on Test Set:
Accuracy: 0.9790
Precision: 0.9782
Recall: 0.9746
F1-Score: 0.9764
```

(a) ResNet50 Output-1

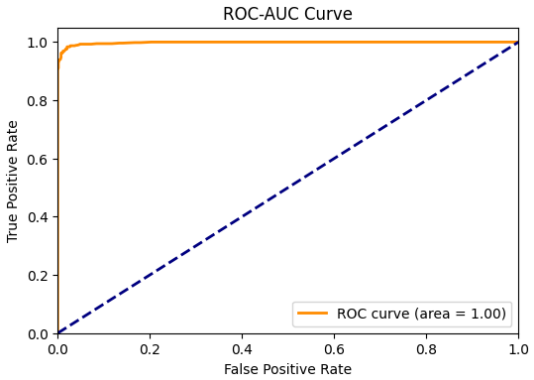
```
ROC-AUC: 0.9981

Class-wise Error Analysis:
NoFire misclassified as Fire: 12 (1.74% of NoFire class)
Fire misclassified as NoFire: 14 (2.54% of Fire class)

Fairness Metrics (Error Rate Balance):
False Positive Rate (NoFire class): 0.0174
False Negative Rate (Fire class): 0.0254
```

(b) ResNet50 Output-2

Figure 4.11: ResNet50 Metrics-1



(a) ResNet50 Confusion Matrix



(b) ResNet50 ROC-AUC

Figure 4.12: ResNet50 Metrics-2

The confusion matrix reflects this high performance; out of 688 "NoFire" samples, 676 were correctly classified, with only 12 misclassified as fire (1.74%) and out of 552 "Fire" samples, 538 were correctly classified, with only 14 misclassified as no-fire (2.54%). These error rates are extremely low and indicate that the model is both sensitive (detects fire reliably) and specific (does not raise unnecessary alarms).

The ROC-AUC score of 0.9981 further illustrates the model's exceptional discriminative power. With a value extremely close to 1.0, ResNet50 + RF is capable of almost perfectly distinguishing between fire and non-fire across different decision thresholds, leaving very little room for ambiguity.

The class-wise error analysis reveals that errors are rare and well balanced across classes. Unlike earlier traditional approaches (e.g., SIFT and ORB), which suffered from high false negative rates, this model managed to minimize both false positives and false negatives to very low levels. The fairness metrics also highlight the strength of this approach. The false positive rate (1.74%) and false negative rate (2.54%) are not only very low but also quite close to each other. This balance ensures that the model does not unfairly prioritize one class over the other. For an application like forest fire detection, this is highly significant: excessive false positives could cause unnecessary alerts and resource wastage, while false negatives could result in undetected fires with catastrophic consequences. The ResNet50 + RF model strikes the right balance between

these two risks.

Overall, the ResNet50-based deep feature extraction with Random Forest classification emerges as the most effective solution among all the methods tested. It provides highly accurate, fair, and discriminative results, making it a strong candidate for real-world deployment in early fire detection systems. This validates the decision to elevate from traditional machine learning feature descriptors to deep learning–based representations, as the latter capture complex patterns in images that handcrafted methods often fail to model adequately.

4.10 Comparative Analysis

The following analysis compares the six experiments on multiple levels: discrimination power (ROC-AUC), balanced performance (F1 and accuracy), the precision/recall trade-off, error distribution patterns (confusion matrices), fairness / operational parity (FPR vs FNR), and practical deployment considerations. The goal is to explain what each result means for forest-fire detection and to draw actionable conclusions for model selection and next steps.

Table 4.2: Performance Comparison of Various Models Across Metrics

Metric	**HOG**	**SIFT**	**GLCM**	**ORB**	**Hybrid**	**ResNet50(DL)**
Accuracy	0.8194	0.8129	0.8653	0.7621	0.8823	**0.9790**
Precision	0.7847	0.8292	0.8360	0.7586	0.8638	**0.9782**
Recall (TPR)	0.8188	0.7301	0.8678	0.6830	0.8732	**0.9746**
F1-score	0.8014	0.7765	0.8516	0.7188	0.8685	**0.9764**
ROC-AUC	0.8908	0.8956	0.9347	0.8492	0.9512	**0.9981**
TN	564	605	594	568	612	676
FP	124	83	94	120	76	12
FN	100	149	73	175	70	14
TP	452	403	479	377	482	538
FP count,%	124 (18.02%)	83 (12.06%)	94 (13.66%)	120 (17.44%)	76 (11.05%)	12 (1.74%)
FN count,%	100 (18.12%)	149 (26.99%)	73 (13.22%)	175 (31.70%)	70 (12.68%)	14 (2.54%)
FPR	0.1802	0.1206	0.1366	0.1744	0.1105	0.0174
FNR	0.1812	0.2699	0.1322	0.3170	0.1268	0.0254

4.10.1 Overall ranking by discriminative ability (ROC-AUC) and final accuracy

1. ResNet50 + RF achieves near-perfect discrimination (AUC = 0.9981) and the highest accuracy (97.90%). This indicates that deep features from a pretrained ResNet50 capture virtually all signal needed to separate fire from non-fire images in this dataset.

2. Fusion (H+S+G+O) + RF is the best among traditional/hybrid handcrafted approaches with AUC = 0.9512 and accuracy 88.23%; it benefits from complementary information across descriptors.

3. GLCM + RF follows (AUC = 0.9347, accuracy 86.53%) — texture features were highly informative for this task.

4. SIFT + RF and HOG + RF are close (AUC 0.8956 and 0.8908 respectively; accuracies 81.3–81.9%), showing respectable discriminative ability but with differing error trade-offs.

5. ORB + RF is weakest (AUC = 0.8492, accuracy 76.21%) among the tested handcrafted descriptors.

AUC orders the models by pure separability independent of thresholds; it is aligned with accuracy in this experiment. The step from handcrafted features → fusion → deep features shows progressively larger gains, suggesting that (a) complementary handcrafted descriptors add value when fused and (b) deep convolutional features capture richer, higher-level patterns than engineered descriptors.

4.10.2 Precision vs Recall trade-offs — what each model prioritizes

Recall (sensitivity) is typically most critical in safety contexts (missed fires are costly); precision controls operational cost of false alarms.

1. ResNet50 + RF: Both precision (97.82%) and recall (97.46%) are extremely high and balanced; the method is simultaneously conservative with false alarms and sensitive to true fires. This is ideal in practice.

2. Fusion (H+S+G+O) + RF and GLCM + RF: Both models show high recall (87.32% and 86.78%, respectively) while maintaining good precision (86.4% and 83.6%). They provide a strong balance and would be suitable where deep models are infeasible.

3. HOG + RF: Balanced (precision 78.47%, recall 81.88%) — a reasonable trade-off and fairly symmetrical error rates.

4. SIFT + RF: High precision (82.92%) but relatively low recall (73.01%). This indicates a tendency to avoid false alarms at the expense of missing a sizeable fraction of fire cases. If false alarms are highly costly and some misses are tolerable, this behavior could be acceptable — but for safety-critical detection, the low recall is problematic.

5. ORB + RF: Both precision and recall are lower (0.7586 and 0.6830), with recall especially poor. ORB's design (binary descriptors, compactness) appears insufficient to reliably capture the relevant fire patterns in this dataset.

If recall is prioritized (typical for fire detection), models should be selected or tuned in that order: ResNet50 > Fusion ≈ GLCM > HOG > SIFT (if threshold adjusted) > ORB . SIFT could be salvaged by threshold lowering (increase sensitivity), but that will reduce precision.

4.10.3 Confusion matrix patterns and class-wise errors — who suffers most?

Comparing FP and FN counts/percentages reveals where each model fails:

1. ResNet50: FP 12 / 688 (1.74%), FN 14 / 552 (2.54%) — extremely low absolute errors and near parity.

2. Fusion: FP 76 (11.05%), FN 70 (12.68%) — errors are low and balanced between the classes.
3. GLCM: FP 94 (13.66%), FN 73 (13.22%) — symmetrical errors; GLCM tends to slightly favor recall.
4. HOG: FP 124 (18.02%), FN 100 (18.12%) — roughly balanced but higher error magnitudes.
5. SIFT: FP 83 (12.06%), FN 149 (26.99%) — skewed toward missing fires (high FN).
6. ORB: FP 120 (17.44%), FN 175 (31.70%) — worst at missing fires and largest absolute FN.

Models that produce high FN percentages (SIFT, ORB) are hazardous for deployment in a detection setting because missed fires are the most costly errors. Models with balanced but higher absolute error rates (HOG) may be acceptable if the misclassification budget is manageable, but ideally the absolute error magnitudes should be reduced as seen in GLCM, Fusion, and especially ResNet50.

4.10.4 Fairness / Error-rate balance (FPR vs FNR) — is performance equitable?

Fairness here is interpreted as operational parity between the cost of false alarms and missed alarms.

- Best parity with low absolute errors: ResNet50 (FPR 0.0174, FNR 0.0254) — nearly equal and minimal.
- Good parity with modest errors: Fusion (FPR 0.1105, FNR 0.1268) and GLCM (0.1366 vs 0.1322). These are balanced and indicate no systematic class bias.
- Balanced but higher errors: HOG (0.1802 vs 0.1812) — parity is present but error magnitudes are larger.
- Biased toward FN (dangerous): SIFT and ORB have substantial imbalance: SIFT FNR 0.2699 vs FPR 0.1206; ORB FNR 0.3170 vs FPR 0.1744.

For deployment where ethical/operational parity matters, Fusion or GLCM are preferable among handcrafted approaches. For strict operational parity and minimal absolute errors, ResNet50 is preferred.

4.10.5 Why certain models performed the way they did?

ResNet50's dominance is explained by its hierarchical, learned filters that can model complex combinations of texture, shape, color, context, and high-level patterns (e.g., flame shapes,

smoke gradients, scene context). Pretraining on ImageNet provides useful priors; global pooling condenses robust channel-wise activations into highly discriminative vectors.
Fusion advantage arises because different descriptors capture complementary signals:

- GLCM captures fine texture (smoke, flame granularity),
- HOG captures edge/shape cues (flame boundaries),
- SIFT/ORB capture local keypoints (distinctive corners/regions).

Concatenation allows the classifier to exploit all cues simultaneously and to compensate when one descriptor is weak for a particular image.
GLCM's strong showing is consistent with the visual nature of fires — texture and second-order intensity statistics are highly informative for distinguishing turbulent flame/smoke from static natural textures.
SIFT and ORB weaknesses (especially high FNs) suggest that interest point detectors/descriptors alone are insufficient in many fire scenes (flames/smoke may not produce stable corner-like keypoints, or descriptors may be inconsistent across scales/illumination), or that averaging descriptors (used here to get fixed-length vectors) diluted discriminative details. SIFT had decent precision because when it does find strong keypoints the signal is trustworthy, but many fire patches lack such points, explaining missed detections.
HOG's middling performance is explained by its ability to capture gradients but not global texture statistics; it gives balanced results but lacks some discriminative depth that GLCM or deep features provide.

4.10.6 In Terms of Computation

The models also varied in terms of computational efficiency, which is important for practical deployment:

- ORB and HOG were computationally the most efficient in terms of feature extraction. ORB uses binary descriptors and is fast, but this comes at the cost of accuracy. HOG is slightly more computationally demanding than ORB but still lightweight, making it suitable for real-time or resource-constrained environments.
- SIFT and GLCM required more time. SIFT, being a scale-invariant feature descriptor, is computationally intensive due to keypoint detection and descriptor formation. GLCM, though simple conceptually, involves computing texture statistics across multiple orientations and distances, which increases feature extraction time, especially for large datasets.
- The Fusion Method significantly increased computational cost, as all four feature extraction processes (HOG, SIFT, GLCM, ORB) had to be executed and their outputs concatenated. Training the Random Forest on such high-dimensional feature vectors also required more time, though still manageable.

- ResNet50 required the highest computational resources, both in feature extraction and model training. Extracting deep features involves running images through a convolutional neural network with millions of parameters. While training the Random Forest on extracted features was relatively fast, the deep feature extraction step dominated the computational cost. However, once features are pre-computed, predictions are fast, making this approach feasible in practice with sufficient GPU resources.

In short, ORB and HOG are fastest but weak in accuracy; SIFT and GLCM are moderately intensive with better performance; Fusion adds overhead but improves results; ResNet50 is the most computationally demanding but also the most accurate.

4.10.7 Robustness and Generalization of the Models

ORB + RF and SIFT + RF were least robust. Their misclassification patterns reveal susceptibility to class imbalance and environmental variability (e.g., variations in lighting, smoke density, or scene clutter). SIFT, despite its invariance properties, failed to capture the high-level discriminative features necessary for reliable fire detection.
HOG + RF provided more stable results, thanks to its sensitivity to edge structures and gradients that often define fire contours and smoke. However, robustness was still limited, with relatively high and balanced error rates.
GLCM + RF was more robust in handling diverse textures and backgrounds. Its reliance on statistical texture information made it better at generalizing across different forest conditions, leading to improved balance between false positives and false negatives.
Fusion Method increased robustness further. By combining complementary features — texture (GLCM), gradient (HOG), scale-invariant points (SIFT), and keypoint efficiency (ORB) — the model was able to generalize better to unseen data. Errors were consistently reduced across both classes, making it more reliable in diverse environments.
ResNet50 + RF proved to be the most robust and generalizable. Deep features captured not just local patterns but also global semantic representations (e.g., flames, smoke, vegetation context). This allowed the model to adapt well across varying conditions, achieving near-perfect ROC-AUC. Its balanced fairness metrics further confirmed its ability to generalize without favoring one class.
Overall, robustness followed the same trend as performance: handcrafted features were prone to environmental variability, fusion mitigated some of these weaknesses, and deep learning provided the strongest generalization across diverse scenarios.

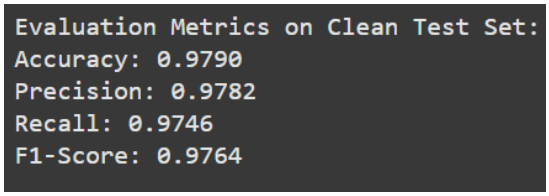
4.10.8 Extended Evaluation of ResNet50

To enhance the robustness evaluation of our fire detection model, we extended the initial experimentation from using ResNet50 for feature extraction on clean images to testing its performance on images corrupted with Gaussian noise. The original setup utilized a pre-trained ResNet50

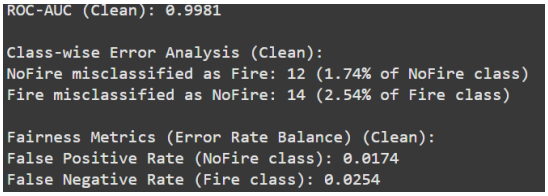
model to extract 2048-dimensional feature vectors from RGB images (resized to 224x224), followed by classification with a Random Forest Classifier. To assess the model's resilience to real-world noise (e.g., sensor noise or environmental distortions), we introduced Gaussian noise (mean=0, standard deviation=25) to the test images. The noisy images were preprocessed similarly to the clean images, and ResNet50 features were extracted for classification. We evaluated both clean and noisy test sets using metrics such as accuracy, precision, recall, F1-score, ROC-AUC, and fairness metrics (FPR and FNR). This extension allowed us to compare the model's performance under ideal and degraded conditions, providing insights into its robustness for practical deployment in fire detection scenarios.

Results

On the clean dataset, ResNet50 combined with Random Forest achieved exceptionally high performance with nearly perfect scores across all metrics: accuracy of 97.9%, F1-score of 97.6%, and an ROC-AUC of 0.9981, highlighting its excellent ability to separate the two classes.
When Gaussian noise was introduced, there was a slight but measurable drop in performance. Accuracy decreased to 97.1% (a decline of 0.8%), precision dropped more sharply to 96.0% (from 97.8%), and F1-score also fell to 96.8%. Interestingly, recall remained unchanged at 97.46%, suggesting that the model's ability to detect fire cases was preserved even in noisy conditions, while precision suffered, reflecting more false alarms.

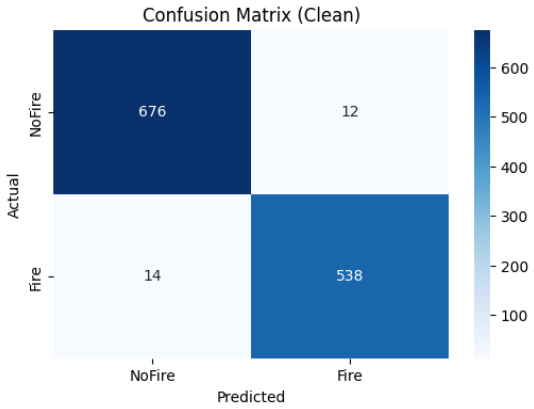


(a) Clean Output-1

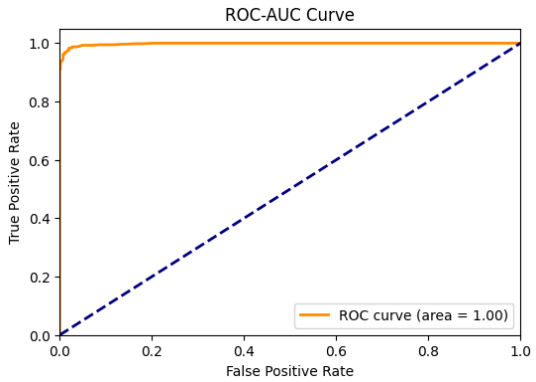


(b) Clean Output-2

Figure 4.13: ResNet50 on Clean Dataset Metrics-1



(a) Clean Confusion Matrix



(b) Clean ROC-AUC

Figure 4.14: ResNet50 on Clean Dataset Metrics-2

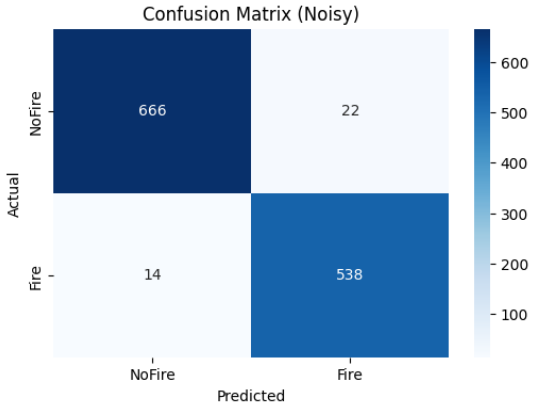
```
Evaluation Metrics on Noisy Test Set:
Accuracy: 0.9710
Precision: 0.9607
Recall: 0.9746
F1-Score: 0.9676
```

(a) Noisy Output-1

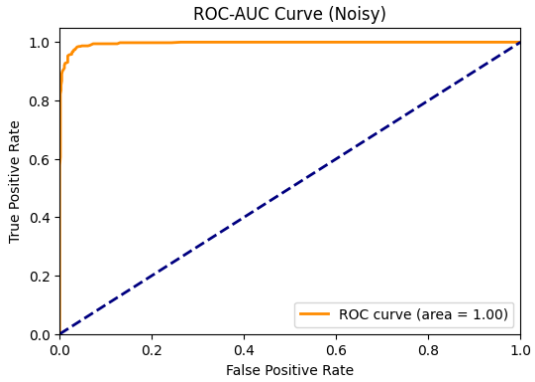
```
ROC-AUC (Noisy): 0.9962

Class-wise Error Analysis (Noisy):
NoFire misclassified as Fire: 22 (3.20% of NoFire class)
Fire misclassified as NoFire: 14 (2.54% of Fire class)

Fairness Metrics (Error Rate Balance) (Noisy):
False Positive Rate (NoFire class): 0.0320
False Negative Rate (Fire class): 0.0254
```

(b) Noisy Output-2

Figure 4.15: ResNet50 on Noisy Dataset Metrics-1



(a) Noisy Confusion Matrix



(b) Noisy ROC-AUC

Figure 4.16: ResNet50 on Noisy Dataset Metrics-2

The confusion matrices reveal the source of these differences. In the clean dataset, 12 NoFire images were misclassified as Fire (false positives), while in the noisy dataset, this number rose to 22. The false negatives (Fire misclassified as NoFire) remained constant at 14 across both datasets. From a fairness perspective, the False Positive Rate (NoFire) nearly doubled under noisy conditions (from 1.74% to 3.20%), while the False Negative Rate (Fire) stayed stable at 2.54%. This indicates that the addition of Gaussian noise disproportionately affects the NoFire class, causing the model to raise more false alarms, while its sensitivity to actual fire events remains intact.

The results provide strong evidence that the ResNet50 + RF pipeline is robust to input perturbations. Despite the deliberate introduction of Gaussian noise, performance degradation was modest (less than 1% drop in accuracy and F1, and $\sim$ 1.5% increase in FPR). The model's recall stability further shows that its detection capacity for true fire cases is resilient to noise. This robustness is critical in real-world applications, where images from drones, satellites, or surveillance systems may be degraded due to environmental factors like smoke, haze, or transmission noise.

On the clean dataset, the model is highly reliable and balanced across both classes. On the noisy dataset, while overall performance remains strong, the increase in false positives means that in a real deployment, slightly more "false fire alerts" would be generated. Although this increases the operational workload, it is still a preferable trade-off compared to missing actual fire events, especially in high-stakes scenarios where early detection is crucial. The findings also

emphasize that data augmentation with noise during training could further improve robustness, reducing false positives in noisy environments.

5. Trade-offs Between Conventional and Deep Learning–Based Feature Extraction Methods

Feature extraction is a crucial stage in any computer vision pipeline, as the quality of features directly determines how well a model can separate classes and generalize to unseen data. Broadly, feature extraction can be carried out using conventional hand-crafted methods or through deep learning–based approaches. Each comes with its own trade-offs, shaped by assumptions about data, computational resources, and the complexity of the task at hand.

Conventional feature extraction methods, such as SIFT, HOG, GLCM, and ORB, are built upon carefully designed mathematical formulations that capture properties like edges, gradients, shapes, or textures. Their strength lies in the fact that they are interpretable and often robust for well-defined tasks. For example, HOG features can effectively capture the shape of objects, while GLCM focuses on texture patterns, making them valuable in domains such as medical imaging or remote sensing where such cues are dominant. Because these descriptors are explicitly engineered, they can perform adequately even on small datasets where training a deep model is infeasible. They are also computationally efficient, making them suitable for real-time systems or environments with limited hardware. However, their limitations become evident when the problem requires capturing higher-level semantics or dealing with highly variable data. Since these features are hand-crafted, they may fail to represent the rich diversity of real-world images, and adapting them to new tasks often requires designing new feature extractors from scratch. Furthermore, they are prone to degradation when faced with noise, lighting variations, or domain shifts that were not anticipated during their design.

Deep learning–based feature extraction methods, by contrast, rely on data-driven learning rather than manually defined rules. Convolutional Neural Networks (CNNs) are the most common example, where layers of convolutional filters learn hierarchical representations directly from raw pixels. The early layers tend to capture low-level patterns such as edges or color contrasts, while deeper layers encode complex shapes, object parts, and even semantic categories. This hierarchical structure allows deep networks to model highly abstract concepts that conventional methods cannot easily capture. Importantly, deep features are learned jointly with the classification task in an end-to-end manner, ensuring that the extracted representations are optimized for the specific objective. Moreover, with transfer learning, pretrained models like ResNet or

VGG can provide high-quality feature representations even when only a small labeled dataset is available. Despite these advantages, deep learning approaches demand significant computational power and large amounts of labeled data during training. They are also less interpretable compared to hand-crafted features, as it is often difficult to precisely explain what information a particular activation encodes. Additionally, they can be sensitive to noise or domain shifts unless trained with strong augmentation or domain adaptation techniques.

The impact of feature representation on model performance cannot be overstated. Effective features increase inter-class separability while reducing intra-class variability, enabling simple classifiers such as linear models to achieve strong performance. Poor features, by contrast, may make classes overlap in the feature space, forcing even complex models to struggle. Robust features also contribute to generalization: a model trained on features that are invariant to noise, illumination, or scaling will perform consistently in real-world scenarios, while fragile features lead to sharp drops in accuracy when conditions change. In terms of efficiency, compact yet discriminative features reduce the risk of overfitting, especially on small datasets, while overly high-dimensional or redundant features may demand more data and regularization.

In practice, the choice between conventional and deep learning–based feature extraction depends on the problem context. For tasks with limited data, constrained computational resources, or strong domain priors, conventional methods may still be preferable. On the other hand, when the goal is to achieve state-of-the-art accuracy on complex and diverse datasets, deep learning approaches are usually the better choice due to their adaptability and representational power. Increasingly, hybrid approaches that combine the interpretability and robustness of conventional descriptors with the expressiveness of deep learning features are being explored. Regardless of the method chosen, the essence remains the same: the quality of feature representation directly governs the performance, generalization, and robustness of machine learning models.

To illustrate this in a real-world setting, consider the task of forest fire detection. A conventional approach using color histograms and texture descriptors can identify fire-like regions by focusing on the distinctive hues of flames and the smoky textures in images. This works reasonably well in controlled scenarios but may fail when faced with noisy data, overcast skies, or objects with similar color profiles. In contrast, a deep learning model such as ResNet50 learns more complex representations that capture not only the appearance of flames but also contextual cues from surrounding vegetation and scene structure. As a result, it tends to outperform conventional methods on diverse datasets, though it may require augmentation strategies to remain robust against noise. This example highlights how the choice of feature representation directly influences a model's ability to balance accuracy, robustness, and generalization in practice.

Bibliography

[1] D. G. Lowe, "Distinctive image features from scale-invariant keypoints," *International Journal of Computer Vision*, vol. 60, no. 2, pp. 91–110, 2004.

[2] N. Dalal and B. Triggs, "Histograms of oriented gradients for human detection," in *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2005, pp. 886–893.

[3] H. Bay, T. Tuytelaars, and L. Van Gool, "Surf: Speeded up robust features," in *European Conference on Computer Vision (ECCV)*. Springer, 2006, pp. 404–417.

[4] E. Rublee, V. Rabaud, K. Konolige, and G. Bradski, "Orb: An efficient alternative to sift or surf," in *International Conference on Computer Vision (ICCV)*. IEEE, 2011, pp. 2564–2571.

[5] M. Calonder, V. Lepetit, C. Strecha, and P. Fua, "Brief: Binary robust independent elementary features," in *European Conference on Computer Vision (ECCV)*. Springer, 2010, pp. 778–792.

[6] G. Csurka, C. R. Dance, L. Fan, J. Willamowski, and C. Bray, "Visual categorization with bags of keypoints," in *Workshop on Statistical Learning in Computer Vision, ECCV*, 2004, pp. 1–22.

[7] H. Jégou, M. Douze, C. Schmid, and P. Pérez, "Aggregating local descriptors into a compact image representation," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2010, pp. 3304–3311.

[8] F. Perronnin, J. Sánchez, and T. Mensink, "Improving the fisher kernel for large-scale image classification," in *European Conference on Computer Vision (ECCV)*. Springer, 2010, pp. 143–156.

[9] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2012, pp. 1097–1105.

[10] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," *arXiv preprint arXiv:1409.1556*, 2014.

[11] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.

[12] K. M. Yi, E. Trulls, V. Lepetit, and P. Fua, "Lift: Learned invariant feature transform," in *European Conference on Computer Vision (ECCV)*. Springer, 2016, pp. 467–483.

[13] D. DeTone, T. Malisiewicz, and A. Rabinovich, "Superpoint: Self-supervised interest point detection and description," in *IEEE Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2018, pp. 224–236.

[14] M. Dusmanu, I. Rocco, T. Pajdla, M. Pollefeys, J. Sivic, A. Torii, and T. Sattler, "D2-net: A trainable cnn for joint detection and description of local features," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 8092–8101.

[15] H. Noh, A. Araujo, J. Sim, T. Weyand, and B. Han, "Large-scale image retrieval with attentive deep local features," in *IEEE International Conference on Computer Vision (ICCV)*, 2017, pp. 3456–3465.

[16] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, "A simple framework for contrastive learning of visual representations," in *International Conference on Machine Learning (ICML)*. PMLR, 2020, pp. 1597–1607.

[17] K. He, H. Fan, Y. Wu, S. Xie, and R. Girshick, "Momentum contrast for unsupervised visual representation learning," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 9729–9738.

[18] J.-B. Grill, F. Strub, F. Altché, C. Tallec, P. H. Richemond, E. Buchatskaya, C. Doersch, B. A. Pires, Z. D. Guo, M. G. Azar *et al.*, "Bootstrap your own latent: A new approach to self-supervised learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020, pp. 21 271–21 284.

[19] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly *et al.*, "An image is worth 16x16 words: Transformers for image recognition at scale," in *International Conference on Learning Representations (ICLR)*, 2020.

[20] Z. Liu, Y. Lin, Y. Cao, H. Hu, Z. Wei, L. Zhang, S. Lin, and B. Guo, "Swin transformer: Hierarchical vision transformer using shifted windows," in *IEEE International Conference on Computer Vision (ICCV)*, 2021, pp. 10 012–10 022.